

CSE 311:

Data Communication

Instructor:

Dr. Md. Monirul Islam

Differential Pulse Code Modulation (DPCM)

PCM and Transmission Channel Bandwidth

Differential pulse code modulation (DPCM)

Predict $\hat{m}[k]$



Calculate the difference, $d[k] = m[k] - \hat{m}[k]$



Represent $d[k]$ in fewer bits and transmit

PCM and Transmission Channel Bandwidth

Differential pulse code modulation (DPCM)

difference betn successive samples is a **special case** of DPCM

In DPCM: $d[k] = m[k] - \hat{m}[k]$

In simple difference: $d[k] = m[k] - m[k-1]$

which assumes $\hat{m}[k] = m[k-1]$

Prediction in DPCM

The key step: predicting the future, a kind of *fortune telling*

Predict $\hat{m}[k]$ from past samples $m[k-1]$, $m[k-2]$, $\dots$

Prediction in DPCM

Assuming that, a signal $m(t)$ has derivatives of all orders at t ,

we can predict $m(t+T_s)$ from $m(t)$

where, T_s is sampling interval

Prediction in DPCM

According to Taylor's series,

$$m(t + T_s) = m(t) + T_s \dot{m}(t) + \frac{T_s^2}{2!} \ddot{m}(t) + \frac{T_s^3}{3!} \dddot{m}(t) + \dots$$

Prediction in DPCM

According to Taylor's series,

$$m(t + T_s) = m(t) + T_s \dot{m}(t) + \frac{T_s^2}{2!} \ddot{m}(t) + \frac{T_s^3}{3!} \dddot{m}(t) + \dots$$

This formula predicts a future sample $m(t+T_s)$ from past sample $m(t)$ and its derivatives

Prediction in DPCM

According to Taylor's series,

$$\begin{aligned} m(t + T_s) &= m(t) + T_s \dot{m}(t) + \frac{T_s^2}{2!} \ddot{m}(t) + \frac{T_s^3}{3!} \dddot{m}(t) + \dots \\ &\approx m(t) + T_s \dot{m}(t) \quad \text{for small } T_s \end{aligned}$$

Prediction in DPCM

According to Taylor's series,

$$\begin{aligned} m(t + T_s) &= m(t) + T_s \dot{m}(t) + \frac{T_s^2}{2!} \ddot{m}(t) + \frac{T_s^3}{3!} \dddot{m}(t) + \dots \\ &\approx m(t) + T_s \dot{m}(t) \quad \text{for small } T_s \end{aligned}$$

To replace with discrete time samples,

Assume, $t = kT_s$

k -th sample $m[k] = m(t) = m(kT_s)$

$(k+1)$ -th sample $m[k+1] = m(t+T_s) = m(kT_s+T_s)$

Prediction in DPCM

According to Taylor's series,

$$\begin{aligned} m(t + T_s) &= m(t) + T_s \dot{m}(t) + \frac{T_s^2}{2!} \ddot{m}(t) + \frac{T_s^3}{3!} \dddot{m}(t) + \dots \\ &\approx m(t) + T_s \dot{m}(t) \quad \text{for small } T_s \end{aligned}$$

And, the derivate is

$$\dot{m}(t) = \left[\frac{m[k] - m[k-1]}{T_s} \right]$$

Prediction in DPCM

According to Taylor's series,

$$\begin{aligned} m(t + T_s) &= m(t) + T_s \dot{m}(t) + \frac{T_s^2}{2!} \ddot{m}(t) + \frac{T_s^3}{3!} \dddot{m}(t) + \dots \\ &\approx m(t) + T_s \dot{m}(t) \quad \text{for small } T_s \end{aligned}$$

Finally,

$$\begin{aligned} m[k + 1] &\approx m[k] + T_s \left[\frac{m[k] - m[k - 1]}{T_s} \right] \\ &= 2m[k] - m[k - 1] \end{aligned}$$

Prediction in DPCM

$$m[k+1] \approx 2m[k] - m[k-1]$$

Prediction in DPCM

$$m[k+1] \approx 2m[k] - m[k-1]$$



$$m[k] \approx 2m[k-1] - m[k-2]$$

Prediction in DPCM

$$m[k+1] \approx 2m[k] - m[k-1]$$



$$m[k] \approx 2m[k-1] - m[k-2]$$

Generalization,

$$m[k] \approx a_1 m[k-1] + a_2 m[k-2] + \cdots + a_N m[k-N]$$

Prediction in DPCM

Generalized prediction,

$$m[k] \approx a_1 m[k-1] + a_2 m[k-2] + \cdots + a_N m[k-N]$$

Prediction in DPCM

Generalized prediction,

$$m[k] \approx a_1 m[k-1] + a_2 m[k-2] + \cdots + a_N m[k-N]$$

The RHS is the predicted value of $m[k]$, that is, $\hat{m}[k]$

Prediction in DPCM

Generalized prediction,

$$m[k] \approx a_1 m[k-1] + a_2 m[k-2] + \cdots + a_N m[k-N]$$

The RHS is the predicted value of $m[k]$, that is, $\hat{m}[k]$

Therefore,

$$\hat{m}[k] = a_1 m[k-1] + a_2 m[k-2] + \cdots + a_N m[k-N]$$

Prediction in DPCM

Predicted output,

$$\hat{m}[k] = a_1 m[k-1] + a_2 m[k-2] + \cdots + a_N m[k-N]$$

Prediction in DPCM

Predicted output,

$$\hat{m}[k] = a_1 m[k-1] + a_2 m[k-2] + \cdots + a_N m[k-N]$$

The most simplified case:

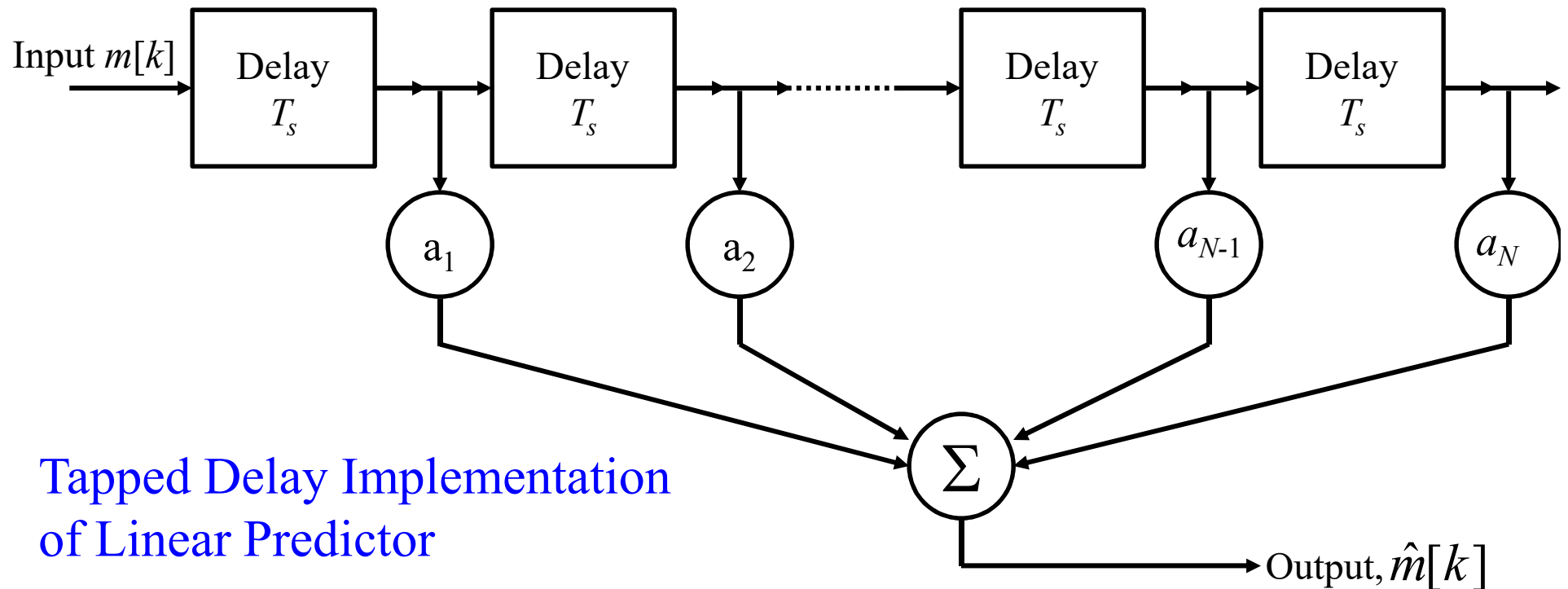
$$\hat{m}[k] = m[k-1]$$

The previous sample is the current sample

Prediction in DPCM

Predicted output,

$$\hat{m}[k] = a_1 m[k-1] + a_2 m[k-2] + \cdots + a_N m[k-N]$$



Prediction in DPCM

Predicted output,

$$\hat{m}[k] = a_1 m[k-1] + a_2 m[k-2] + \cdots + a_N m[k-N]$$

- We **DO NOT** transmit $m[k]$
- We transmit **difference** betn $m[k]$ and $\hat{m}[k]$

$$d[k] = m[k] - \hat{m}[k]$$

- At receiver, we **recalculate** $m[k]$ from $d[k]$ and $\hat{m}[k]$

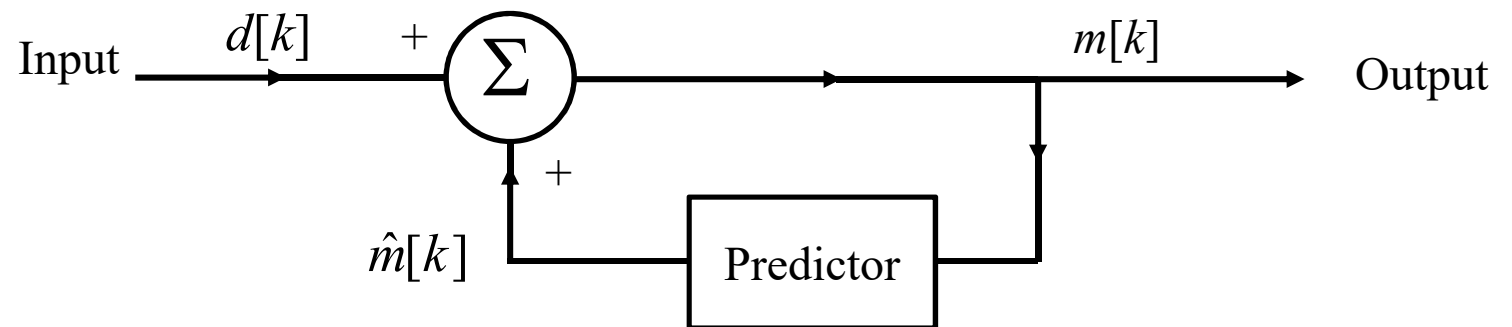
$$m[k] = d[k] + \hat{m}[k]$$

Prediction in DPCM

Transmitter



Receiver



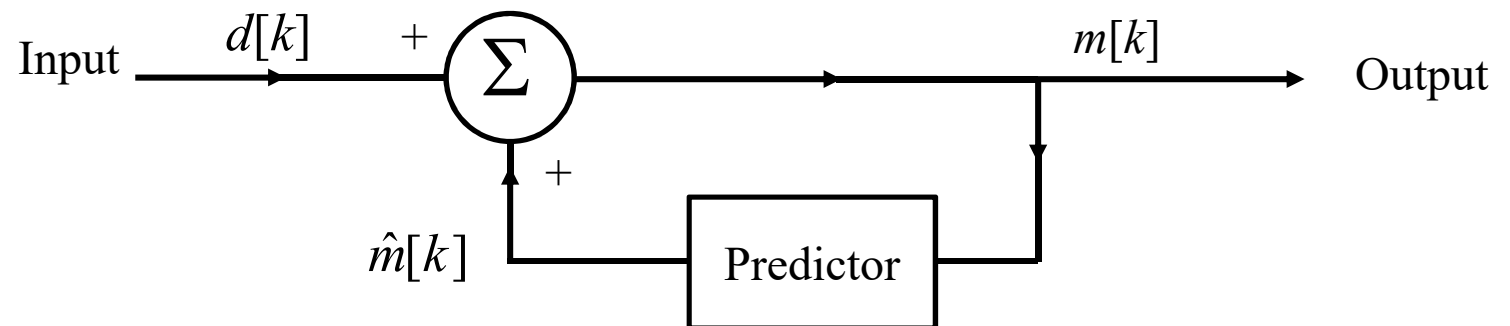
Prediction in DPCM

Transmitter



At Receiver:

$$\begin{aligned} m[k] &= \hat{m}[k] + d[k] \\ &= a_1 m[k-1] + a_2 m[k-2] + \cdots + a_N m[k-N] + d[k] \end{aligned}$$



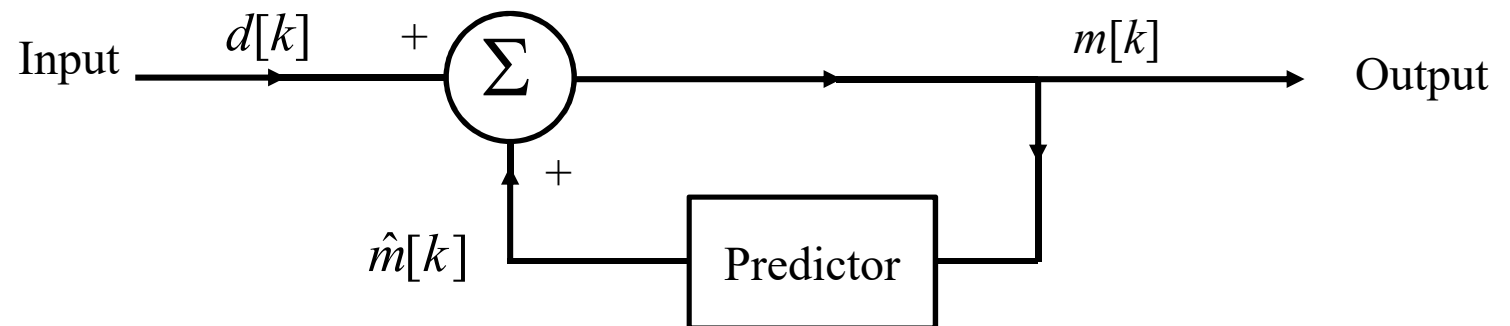
Prediction in DPCM

Transmitter



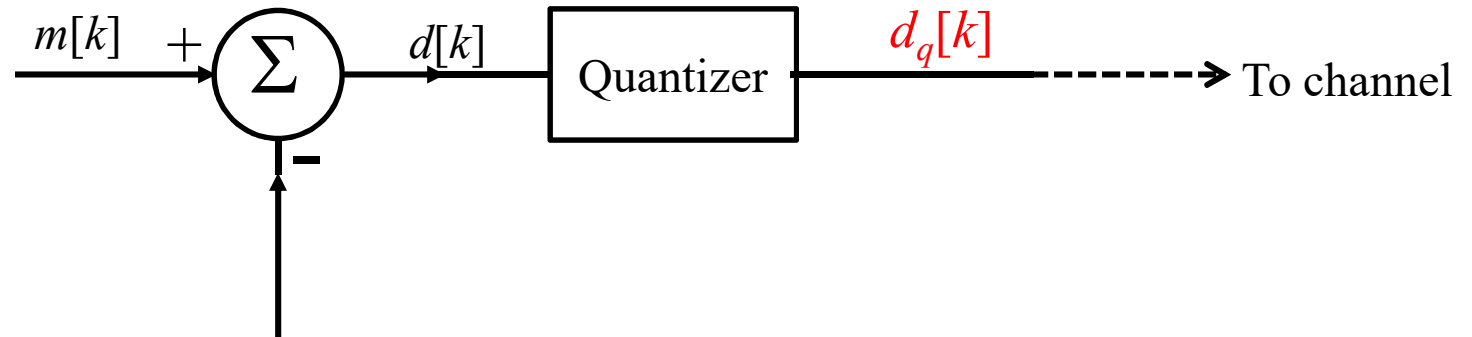
Difficulty at Transmitter:

We **DO NOT** transmit $d[k]$, rather its **quantized** version $d_q[k]$



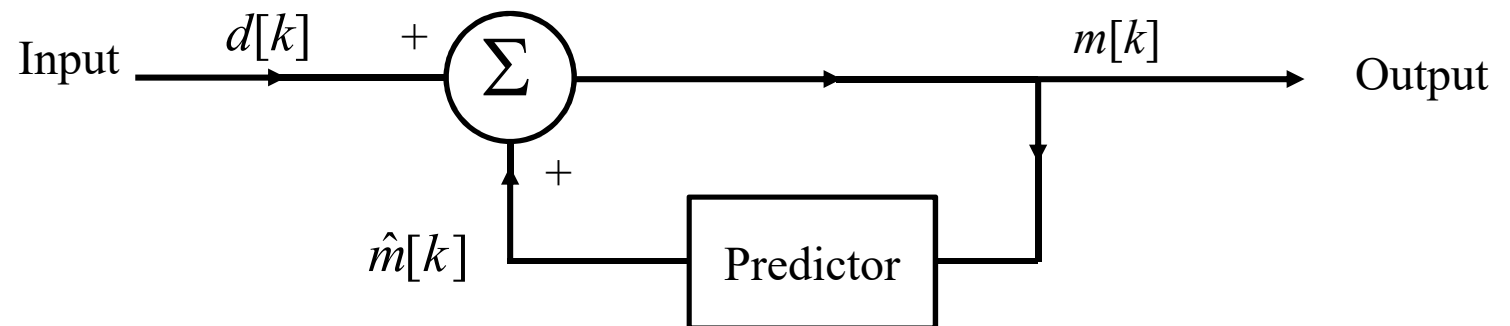
Prediction in DPCM

Transmitter



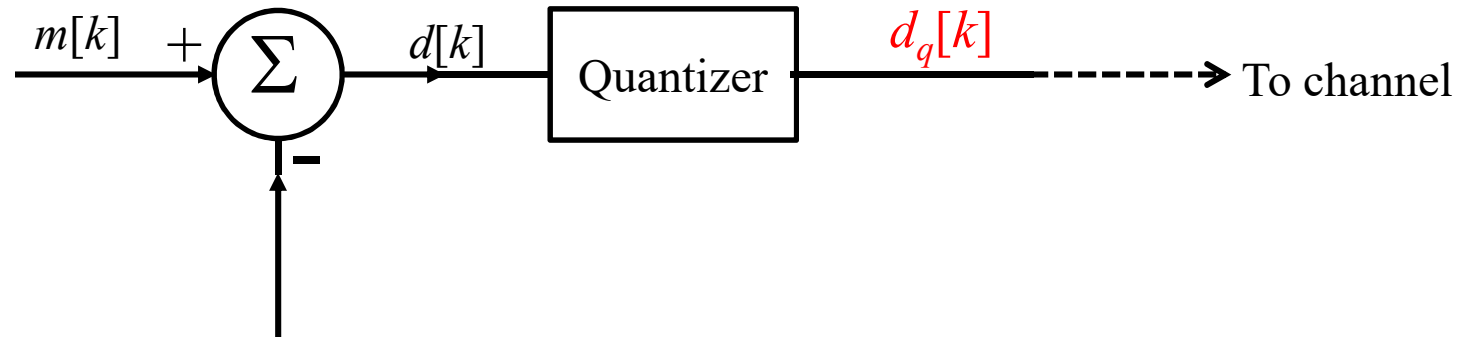
Difficulty at Transmitter:

We **DO NOT** transmit $d[k]$, rather its **quantized** version $d_q[k]$

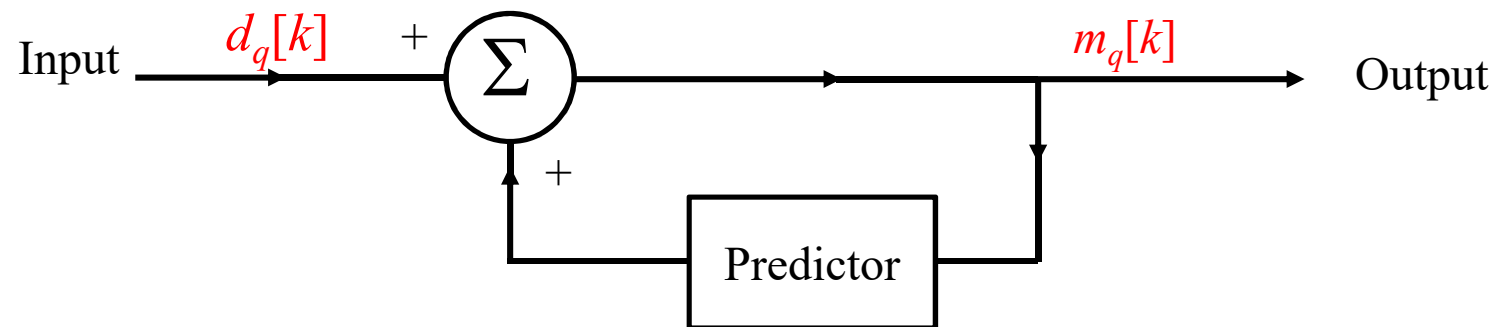


Prediction in DPCM

Transmitter

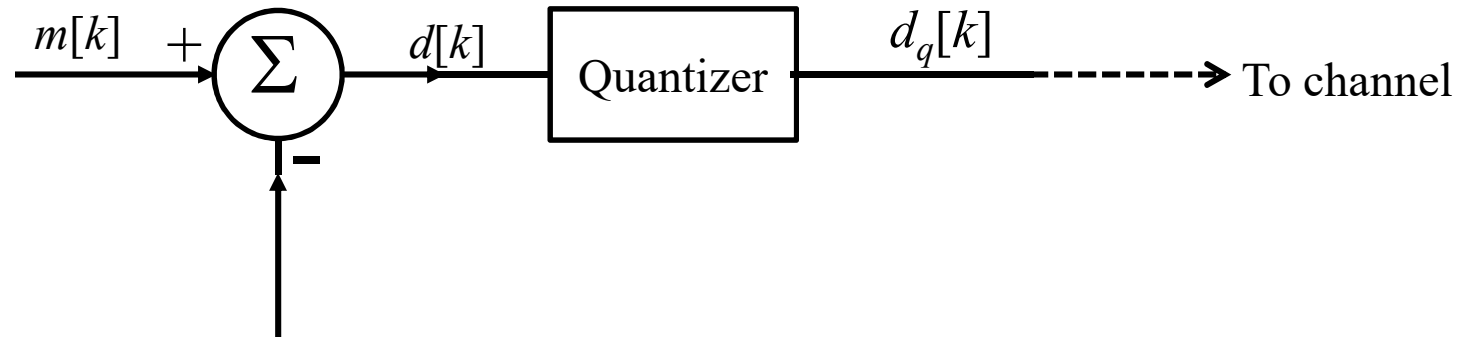


Receiver



Prediction in DPCM

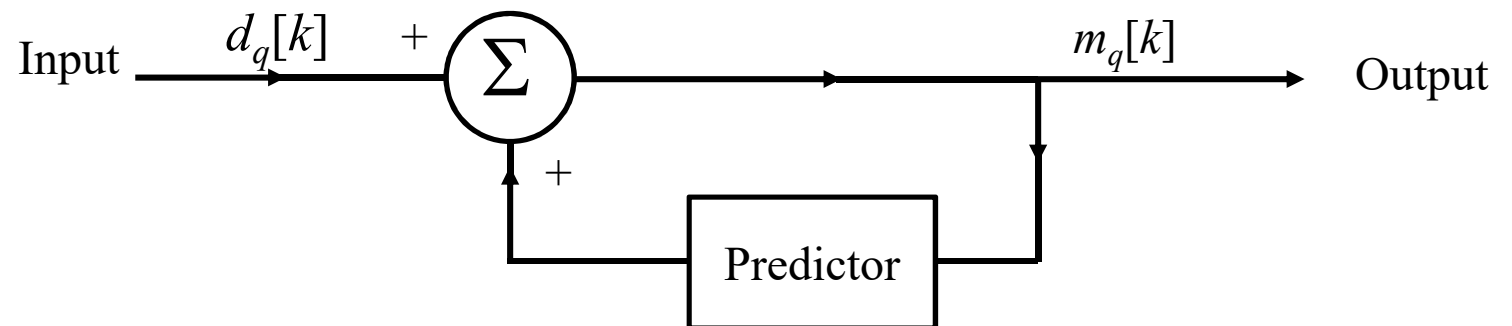
Transmitter



Predictor will generate

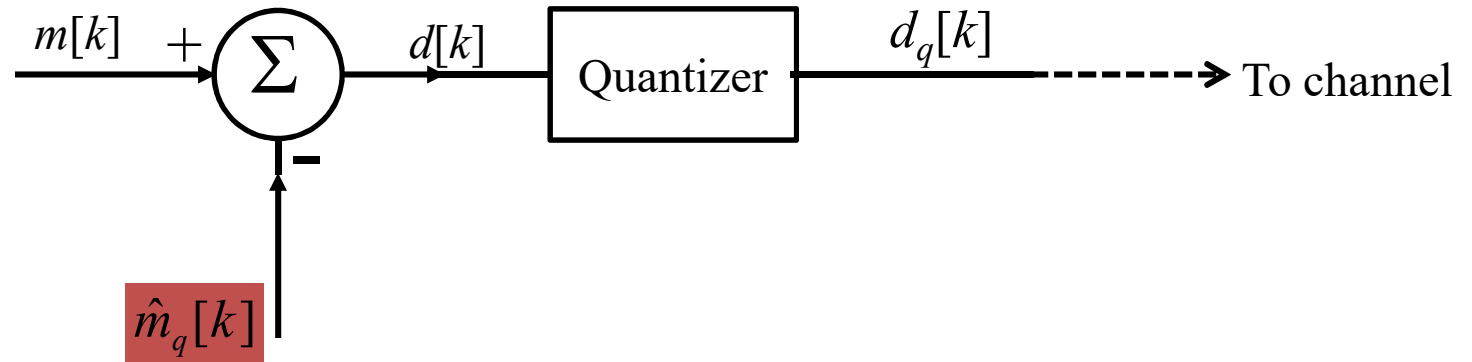
$$\hat{m}_q[k] = a_1 m_q[k-1] + a_2 m_q[k-2] + \dots + a_N m_q[k-N]$$

Receiver



Prediction in DPCM

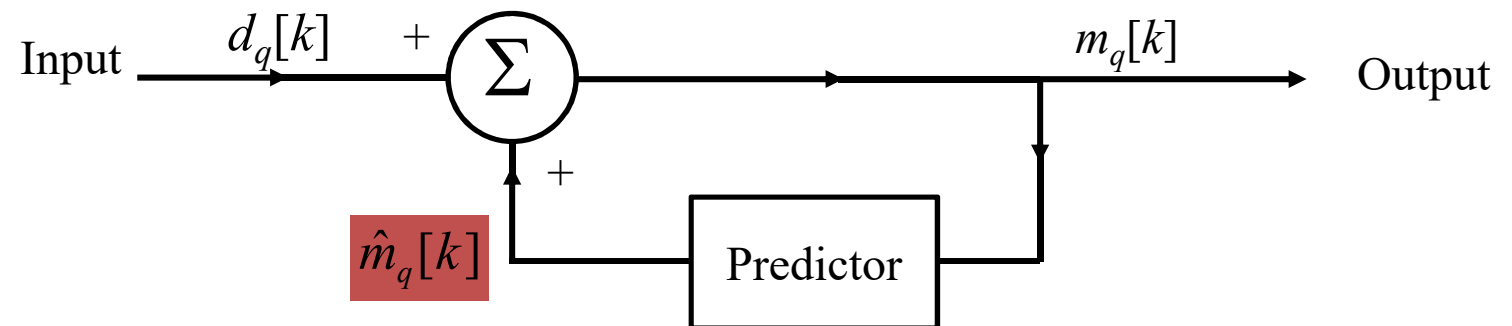
Transmitter



Predictor will generate

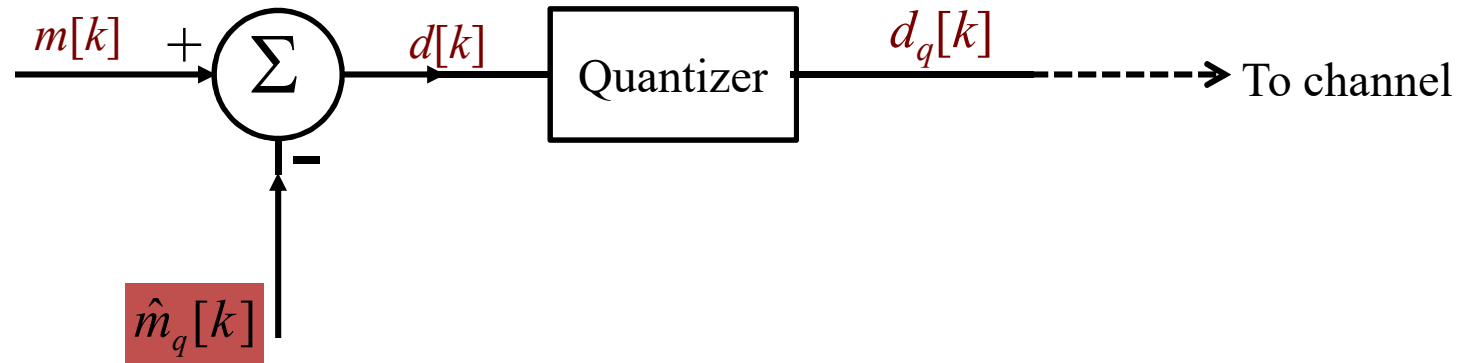
$$\hat{m}_q[k] = a_1 m_q[k-1] + a_2 m_q[k-2] + \cdots + a_N m_q[k-N]$$

Receiver



Prediction in DPCM

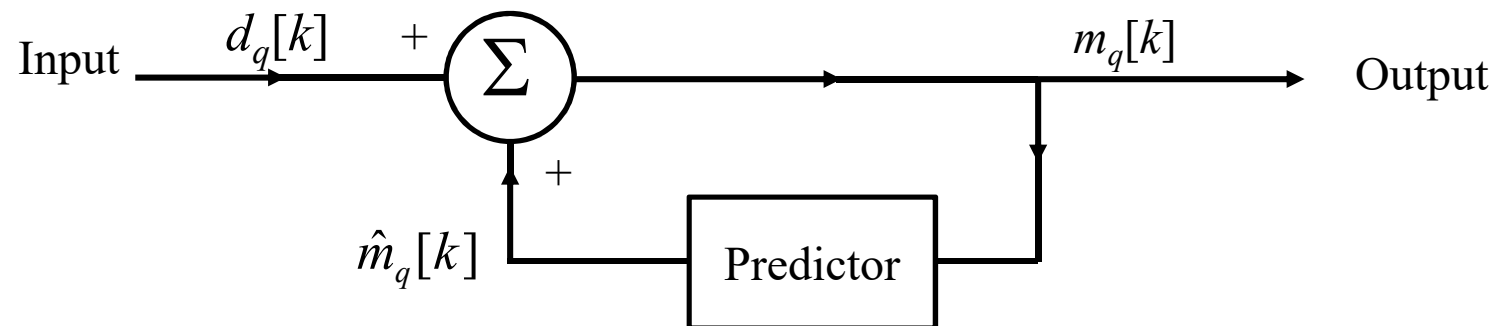
Transmitter



The difference $d[k] = m[k] - \hat{m}_q[k]$ is quantized to yield $d_q[k]$:

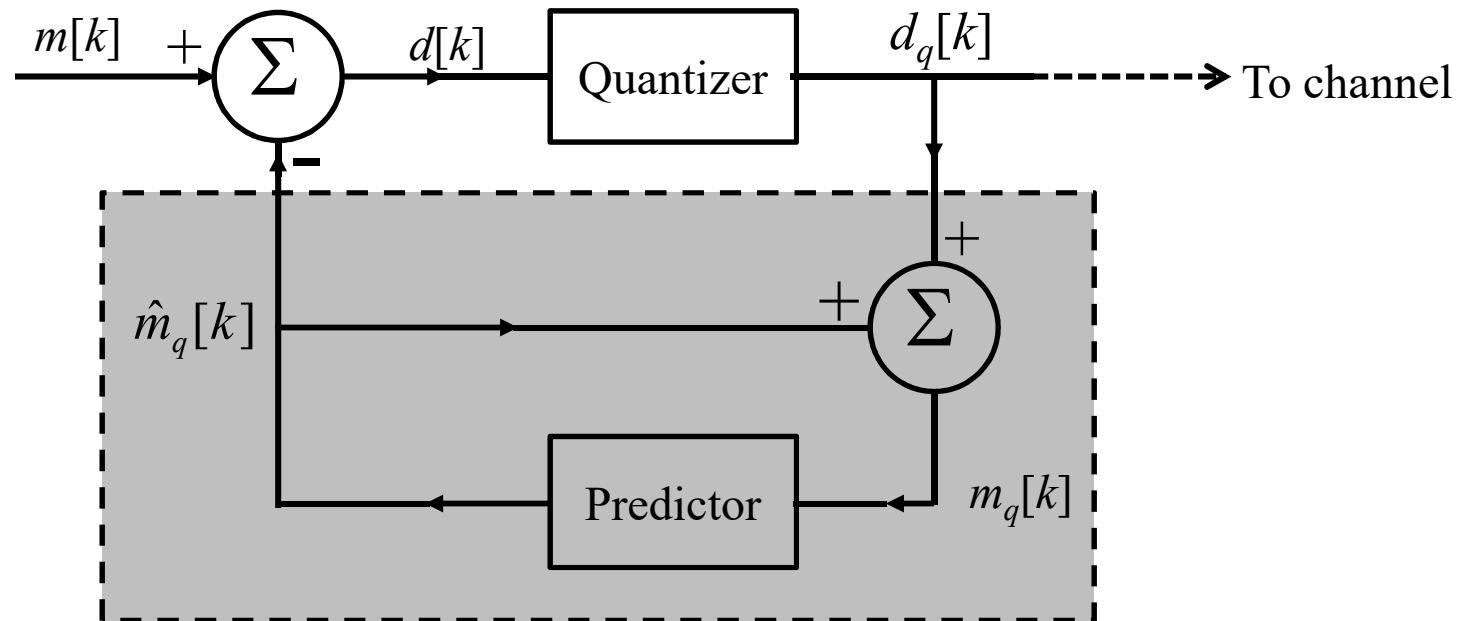
$d_q[k] = d[k] + q[k]$, where $q[k]$ is quantization error

Receiver

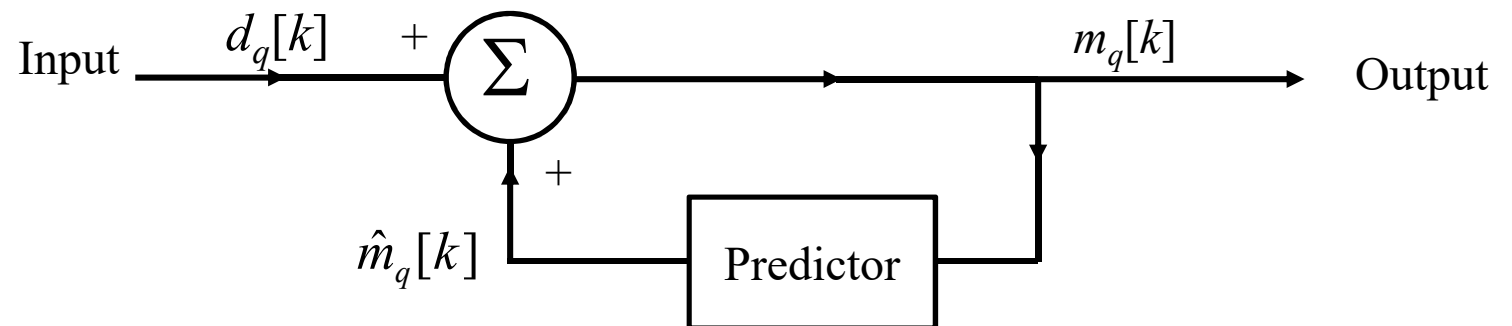


Prediction in DPCM

Transmitter

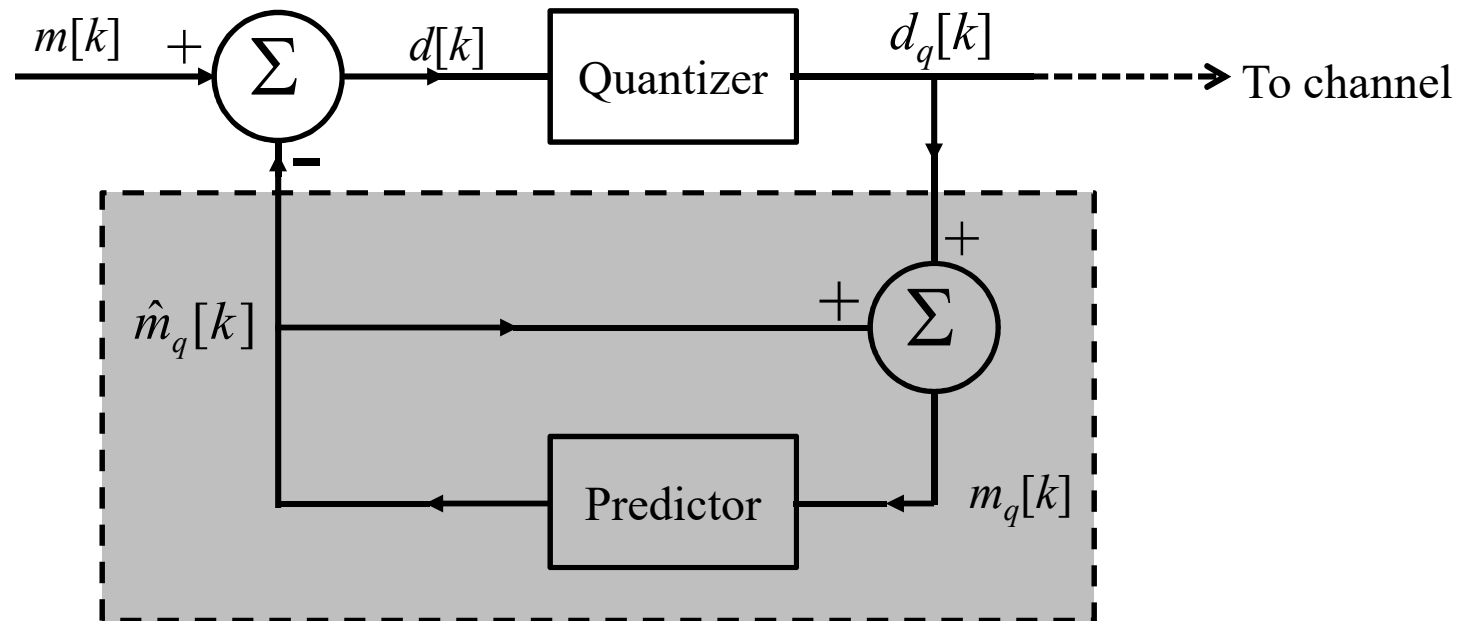


Receiver



Prediction in DPCM

Transmitter

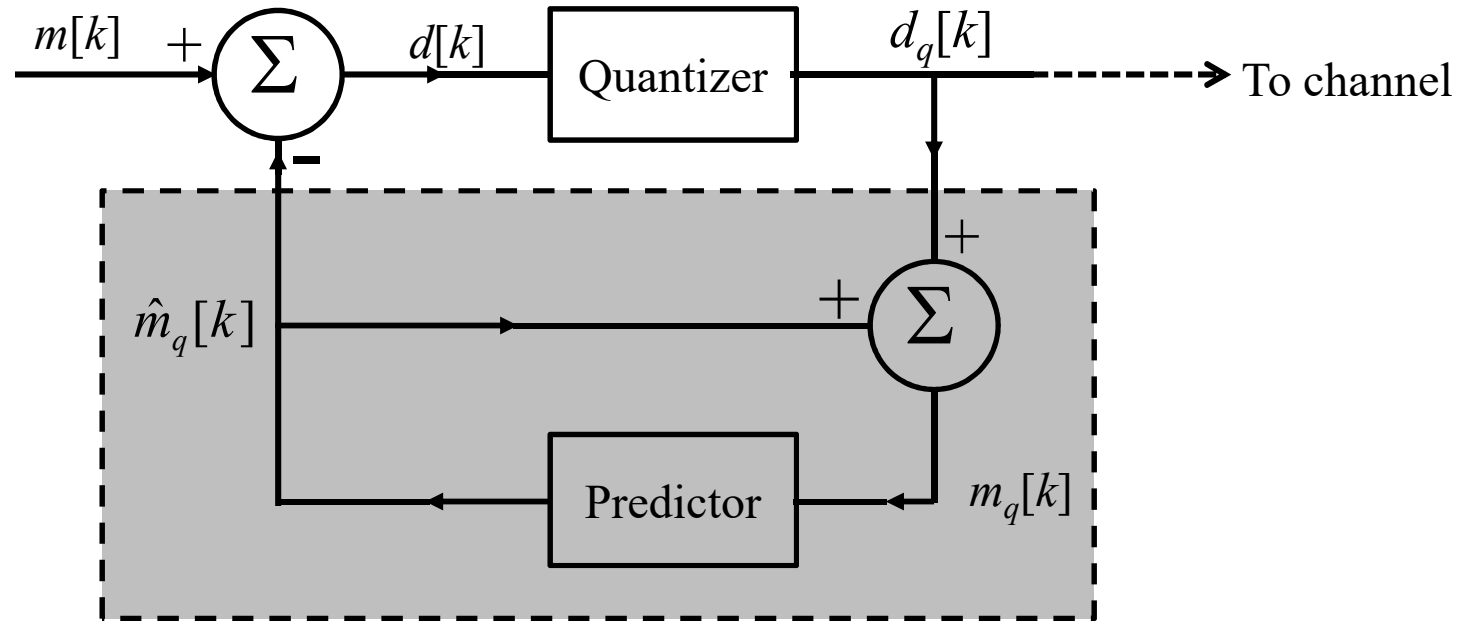


Predictor output $\hat{m}_q[k]$ is feedbacked to its input, that is,

$$m_q[k] = \hat{m}_q[k] + d_q[k]$$

Prediction in DPCM

Transmitter

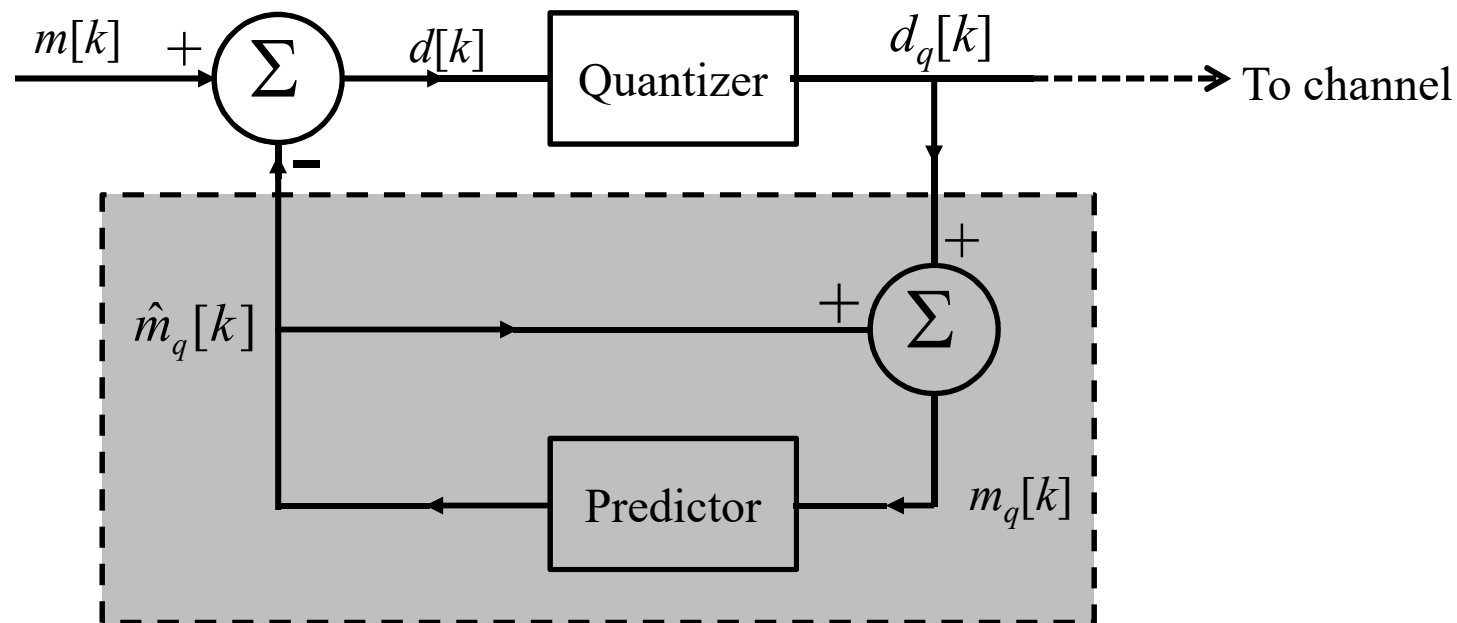


Predictor output $\hat{m}_q[k]$ is feedbacked to its input, that is,

$$\begin{aligned} m_q[k] &= \hat{m}_q[k] + d_q[k] \\ &= m[k] - d[k] + d_q[k] \quad \text{because, } d[k] = m[k] - \hat{m}_q[k] \end{aligned}$$

Prediction in DPCM

Transmitter



Predictor output $\hat{m}_q[k]$ is feedbacked to its input, that is,

$$\begin{aligned} m_q[k] &= \hat{m}_q[k] + d_q[k] \\ &= m[k] - d[k] + d_q[k] \quad \text{because, } d[k] = m[k] - \hat{m}_q[k] \\ &= m[k] + q[k] \quad \text{because, } d_q[k] = d[k] + q[k] \end{aligned}$$

SNR Improvement in DPCM

Let,

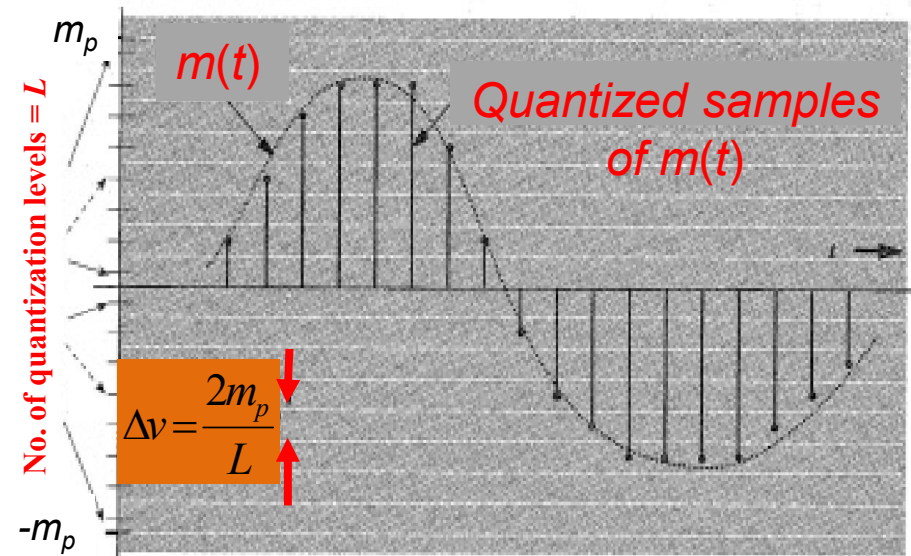
m_p = peak value of message signal $m(t)$

d_p = peak value of difference signal $d(t)$

We know,

$$\Delta v_m = \text{quantization step of } m(t) = \frac{2m_p}{L}$$

$$\Delta v_d = \text{quantization step of } d(t) = \frac{2d_p}{L}$$



Therefore, quantization step **reduction** by a factor of $\frac{d_p}{m_p}$

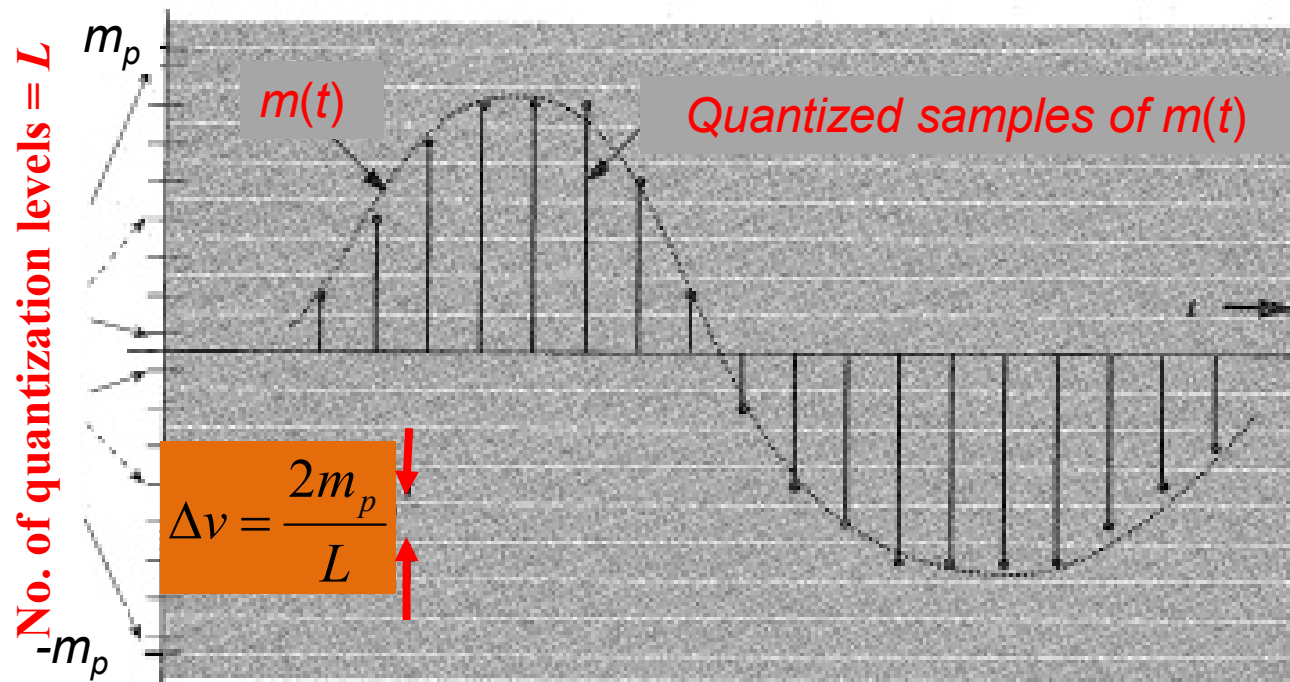
As quantization noise $N_0 = (\Delta v)^2/12$, it reduces by a factor of $\left(\frac{d_p}{m_p}\right)^2$

Improvement of DPCM: Delta Modulation

Improvement of DPCM: Delta Modulation

DPCM Motivation:

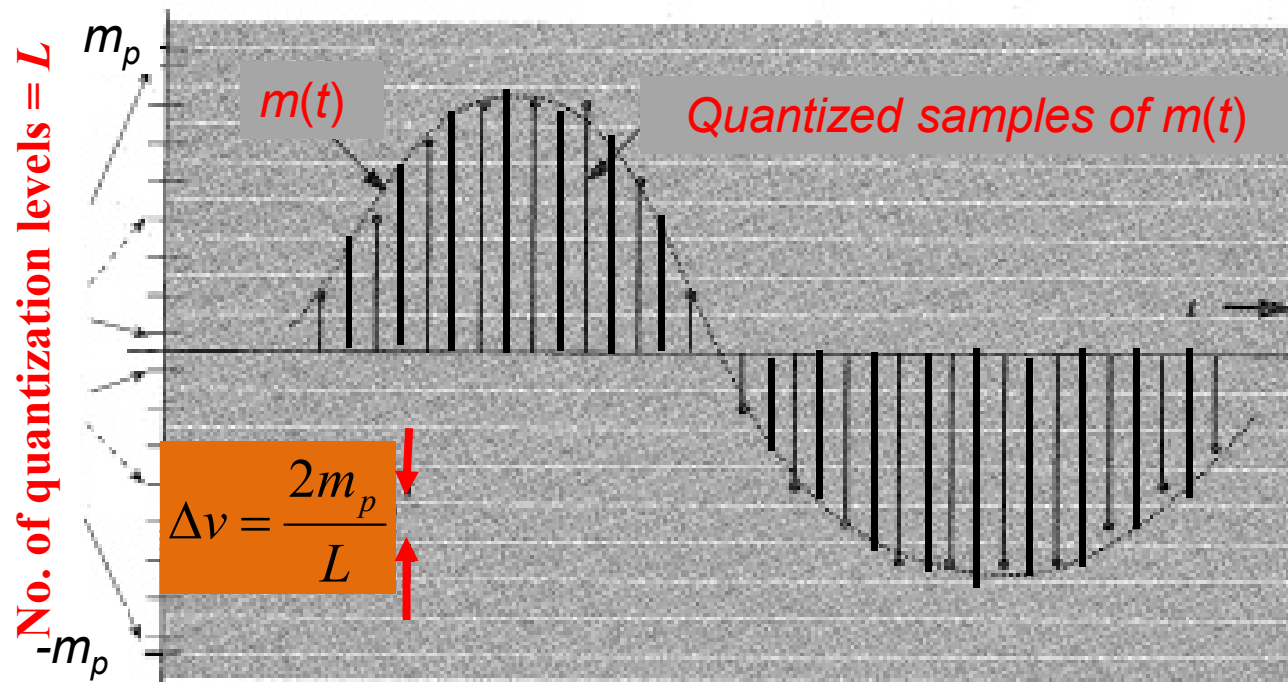
- Successive samples are NOT independent, can be predicted
- Redundancy (CORRELATION) exists among neighboring samples
- Difference can be transmitted instead of actual sample



Improvement of DPCM: Delta Modulation

DPCM Motivation:

- Successive samples are NOT independent, can be predicted
- Redundancy (CORRELATION) exists among neighboring samples
- Difference can be transmitted instead of actual sample

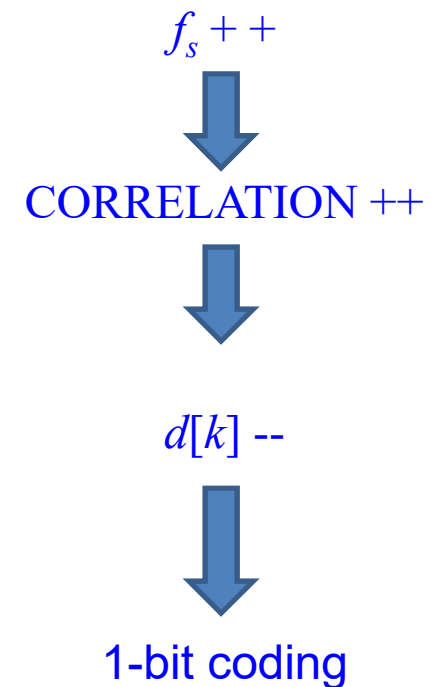
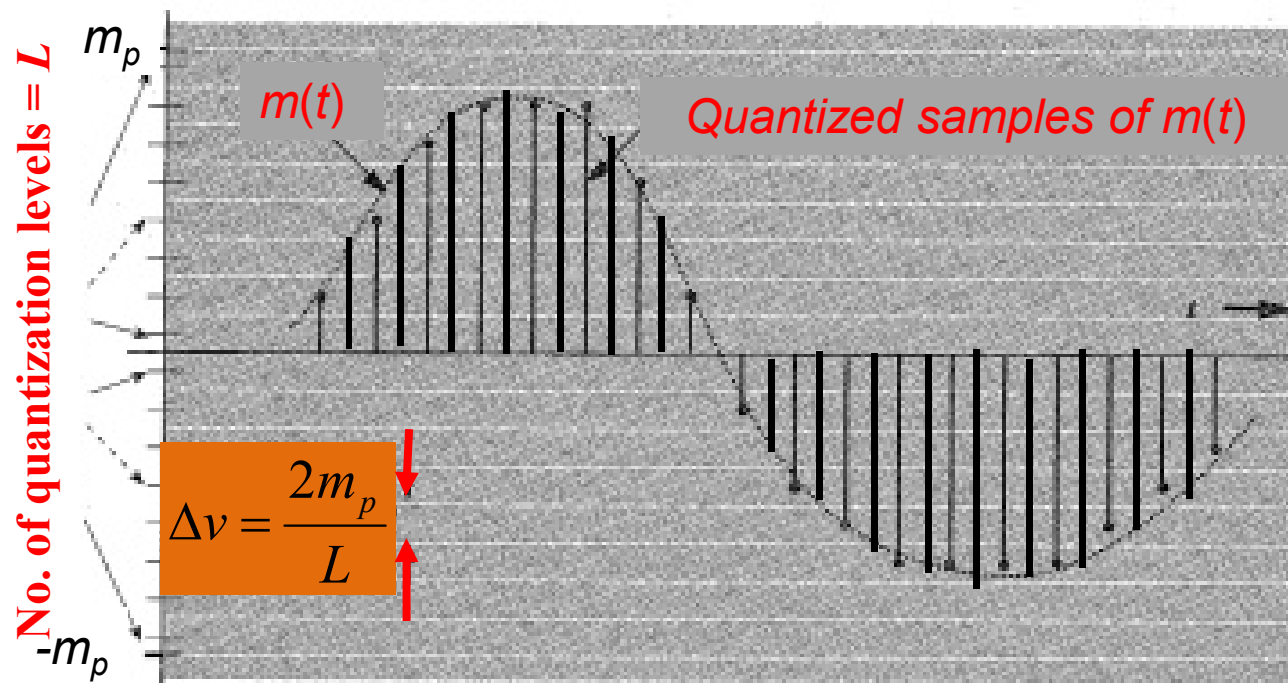


CORRELATION
further increases
if samples are
taken too closely

Improvement of DPCM: Delta Modulation

DPCM Motivation:

- Successive samples are NOT independent, can be predicted
- Redundancy (CORRELATION) exists among neighboring samples
- Difference can be transmitted instead of actual sample



Improvement of DPCM: Delta Modulation

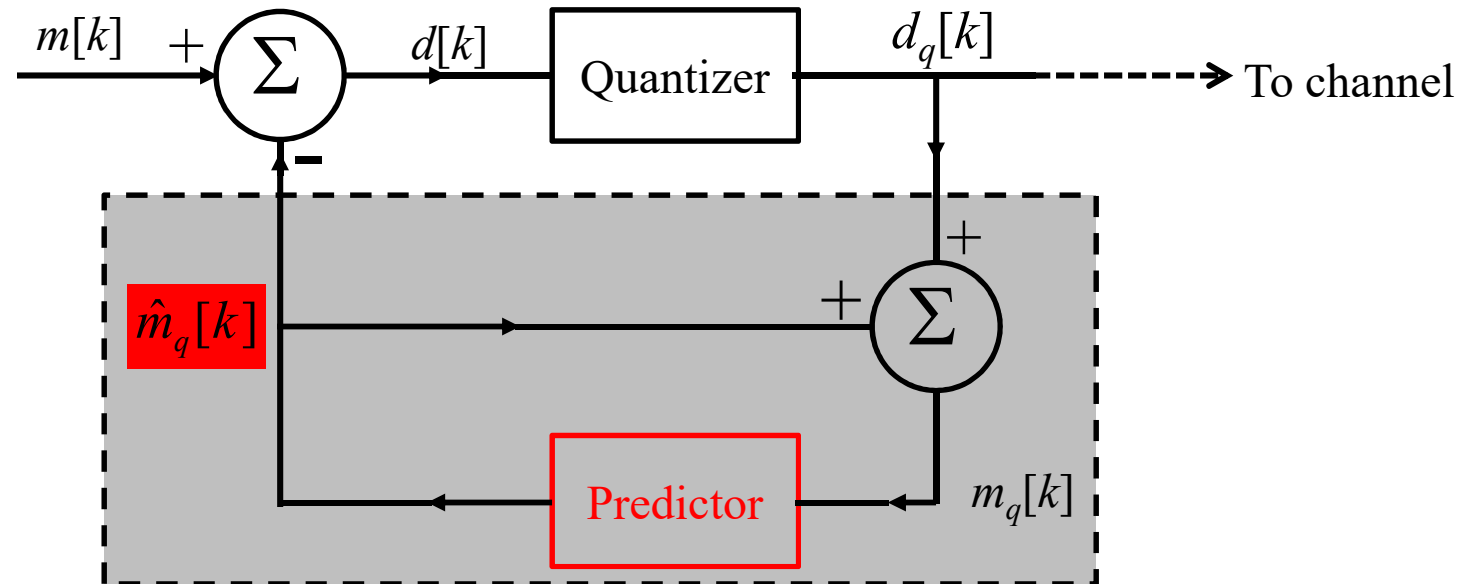
Delta Modulation:

- 1-bit DPCM
- Difference $m[k] - \hat{m}_q[k]$ is quantized in $L = 2$ levels
- Very inexpensive A/D conversion
- $\hat{m}_q[k]$ is assumed to be the previous sample $m_q[k-1]$

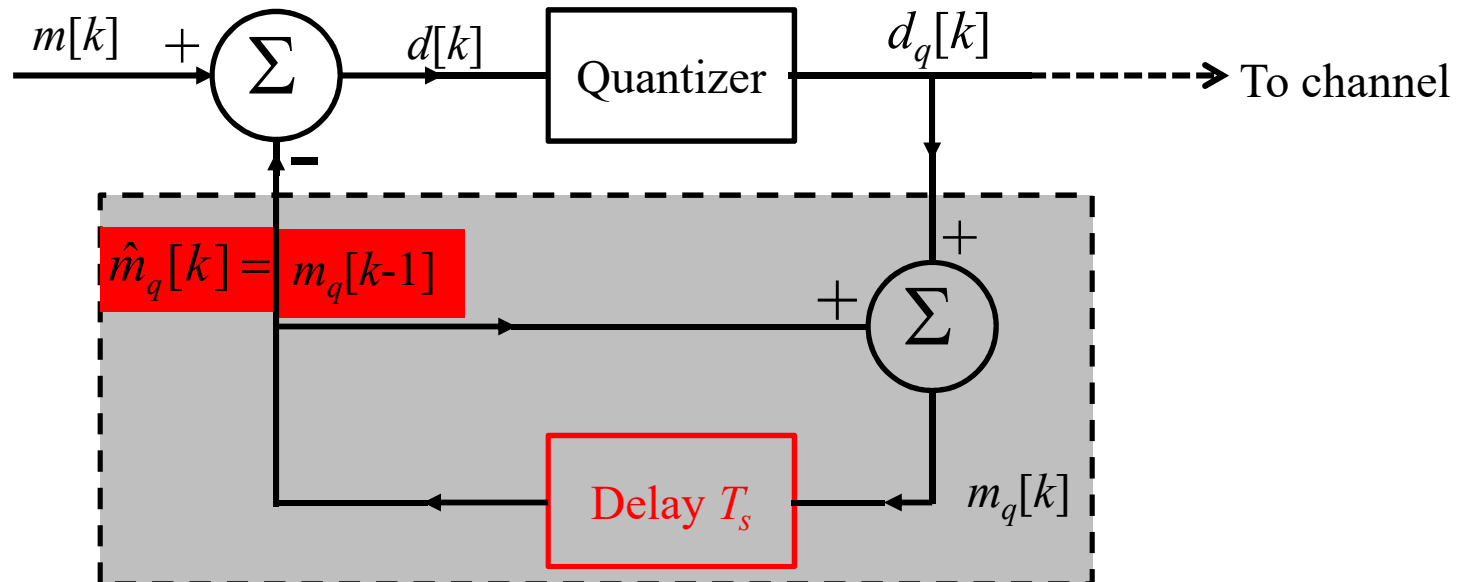
$$\hat{m}_q[k] = m_q[k-1]$$

Transmitters: DPCM Vs. DM

DPCM

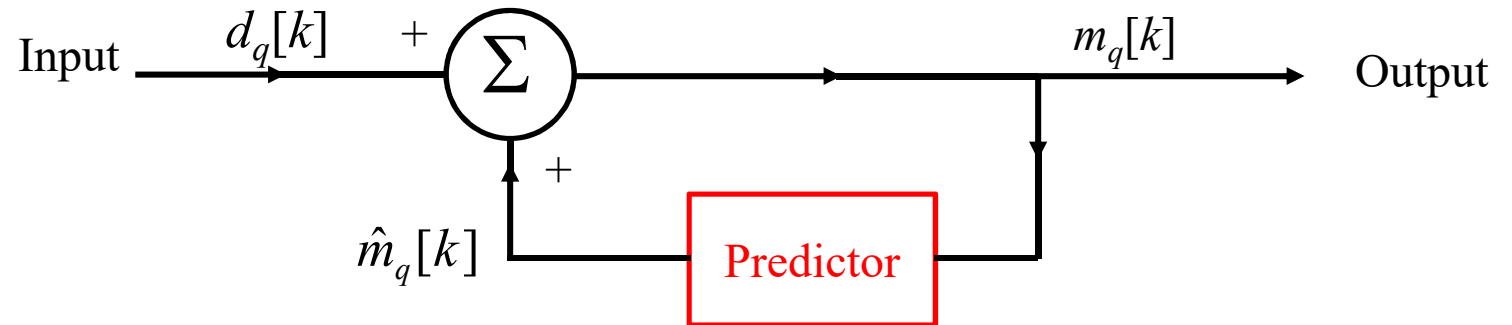


DM

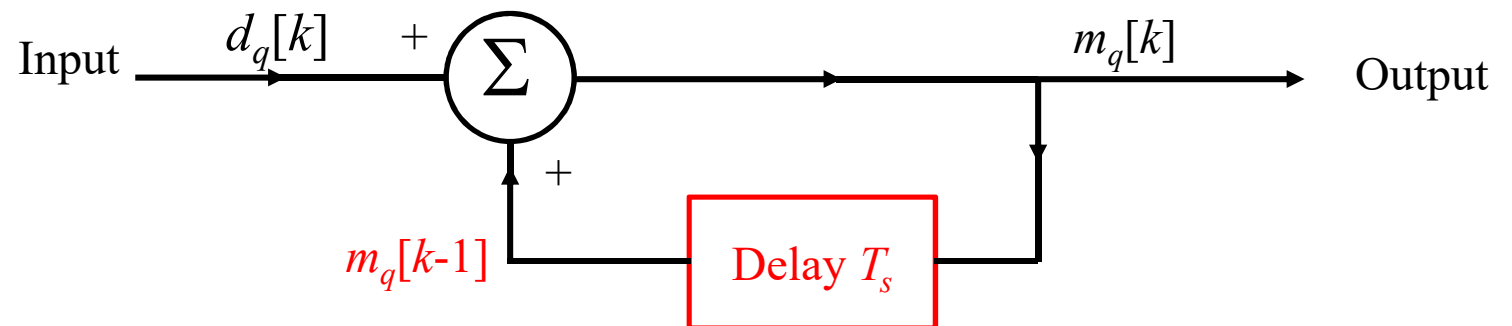


Receivers: DPCM Vs. DM

DPCM

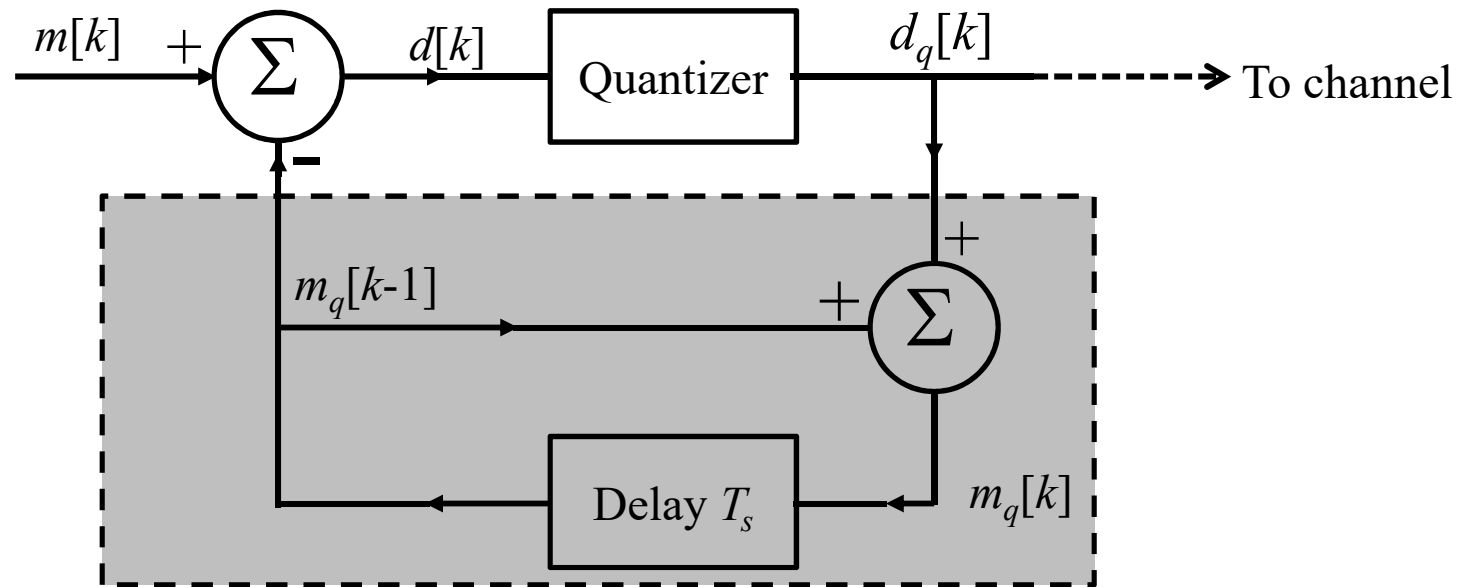


DM

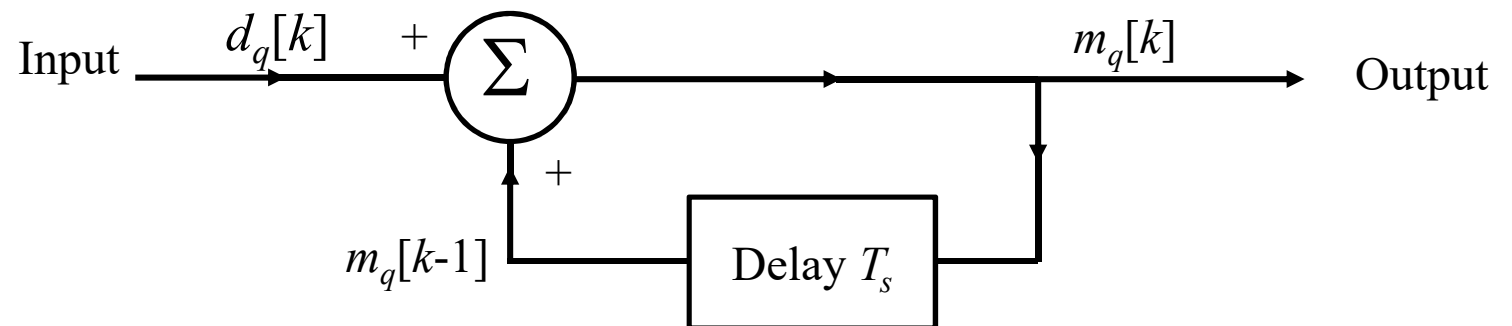


Delta Modulation

Transmitter

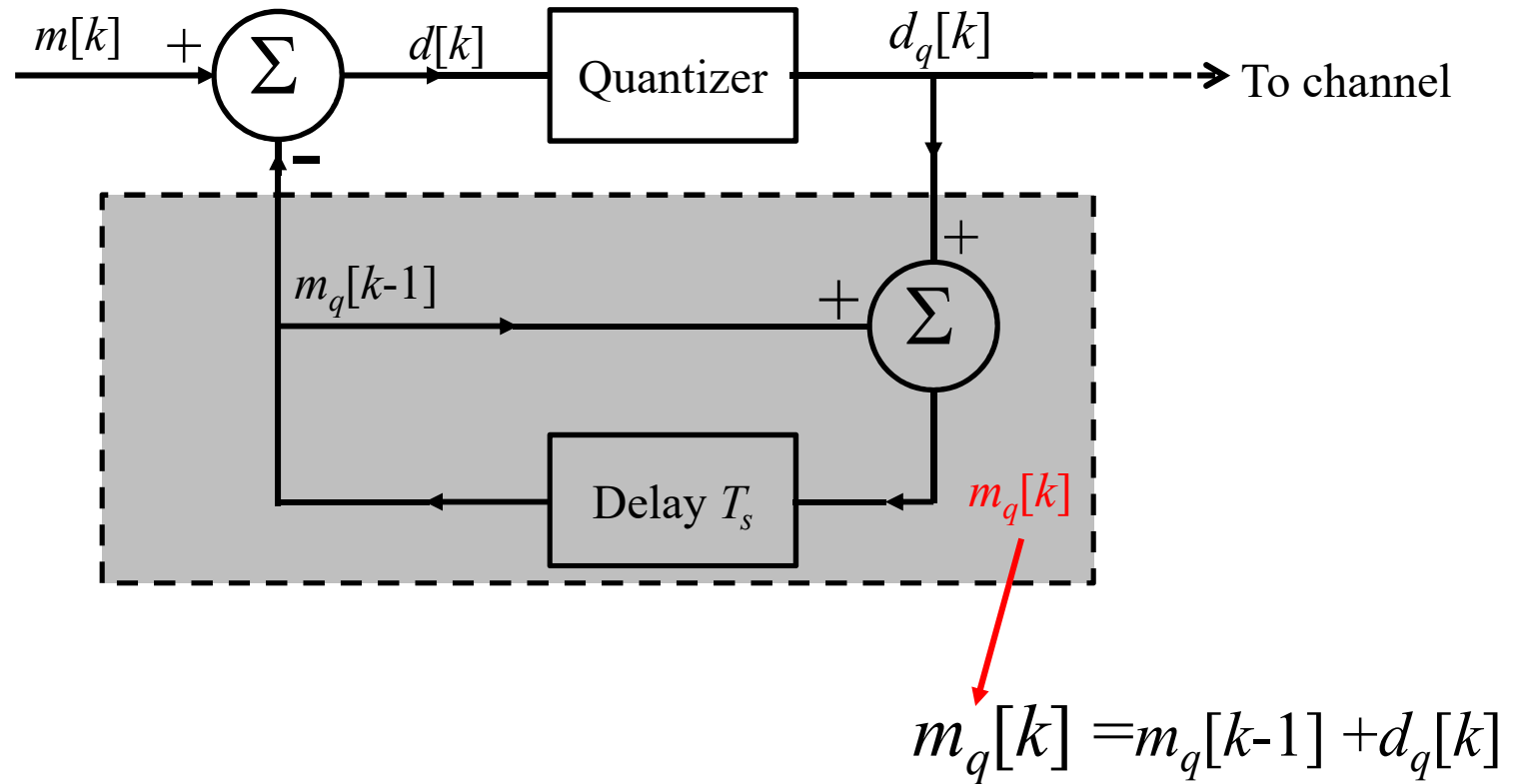


Receiver

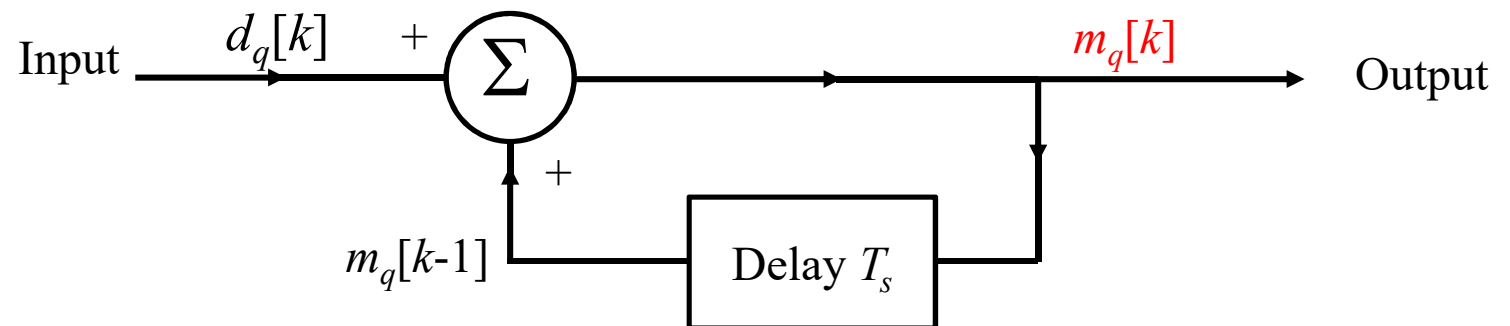


Delta Modulation

Transmitter



Receiver

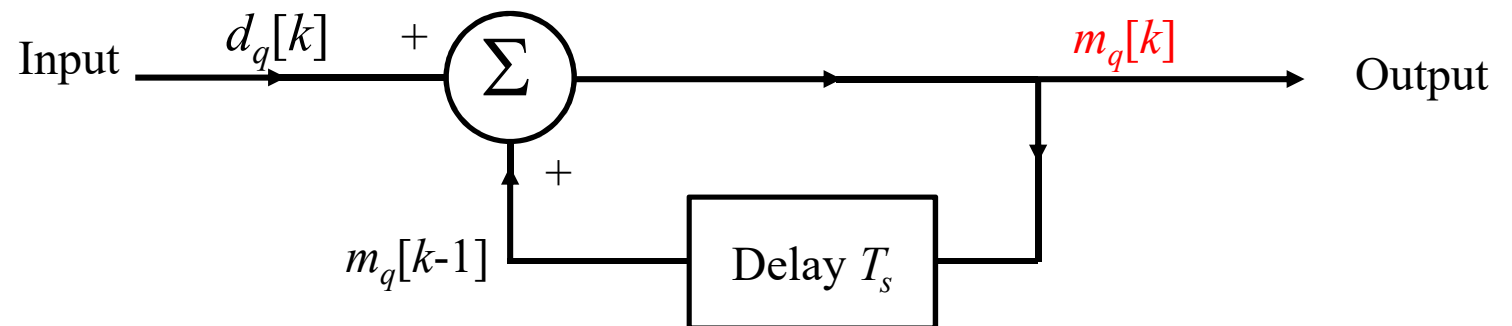


Delta Modulation

$$m_q[k-1] = m_q[k-2] + d_q[k-1]$$

$$m_q[k] = m_q[k-1] + d_q[k]$$

Receiver



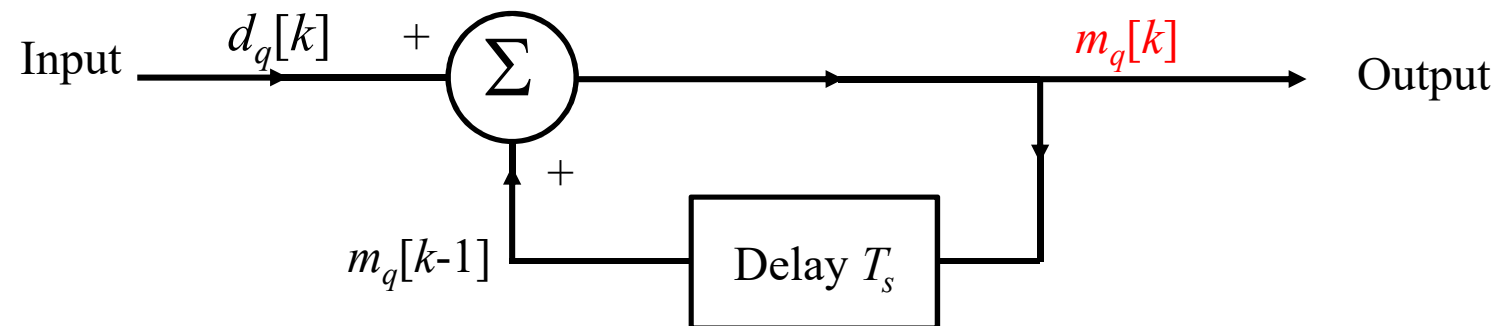
Delta Modulation

$$m_q[k] = m_q[k-2] + d_q[k] + d_q[k-1]$$

$$m_q[k-1] = m_q[k-2] + d_q[k-1]$$

$$m_q[k] = m_q[k-1] + d_q[k]$$

Receiver



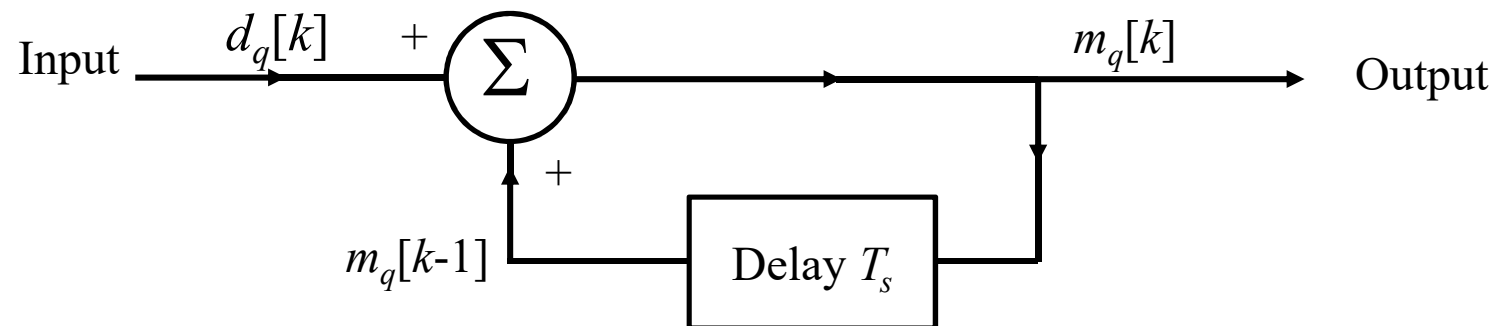
Delta Modulation

$$m_q[k] = m_q[k-2] + d_q[k] + d_q[k-1]$$

Finally, we get

$$m_q[k] = \sum_{m=0}^k d_q[m]$$

Receiver



Delta Modulation

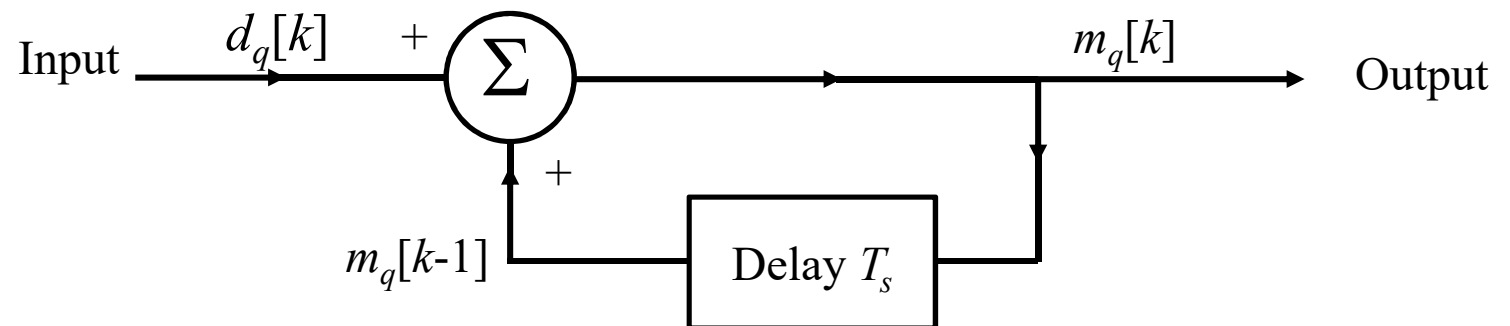
$$m_q[k] = m_q[k-2] + d_q[k] + d_q[k-1]$$

Finally, we get

$$m_q[k] = \sum_{m=0}^k d_q[m]$$

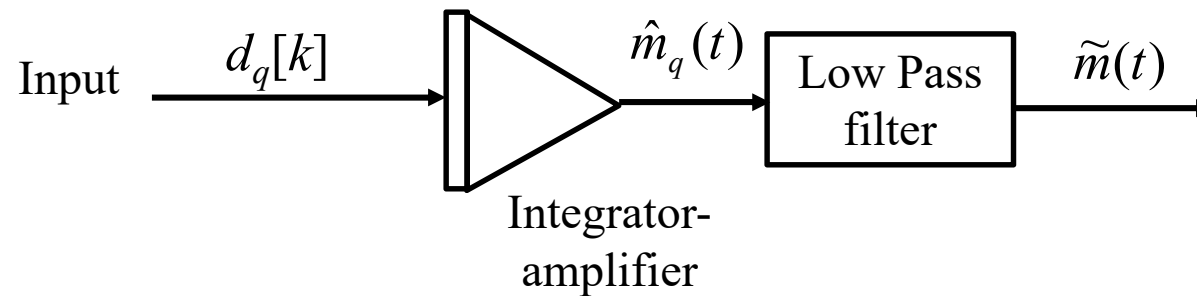
That means, $m_q[k]$ is generated accumulating $d_q[k]$'s

Receiver

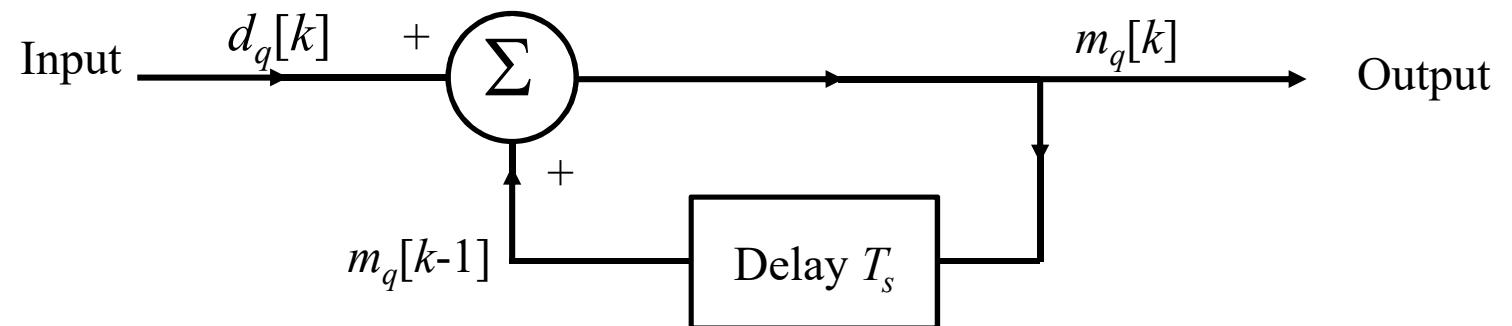


Delta Modulation

Alternate Receiver

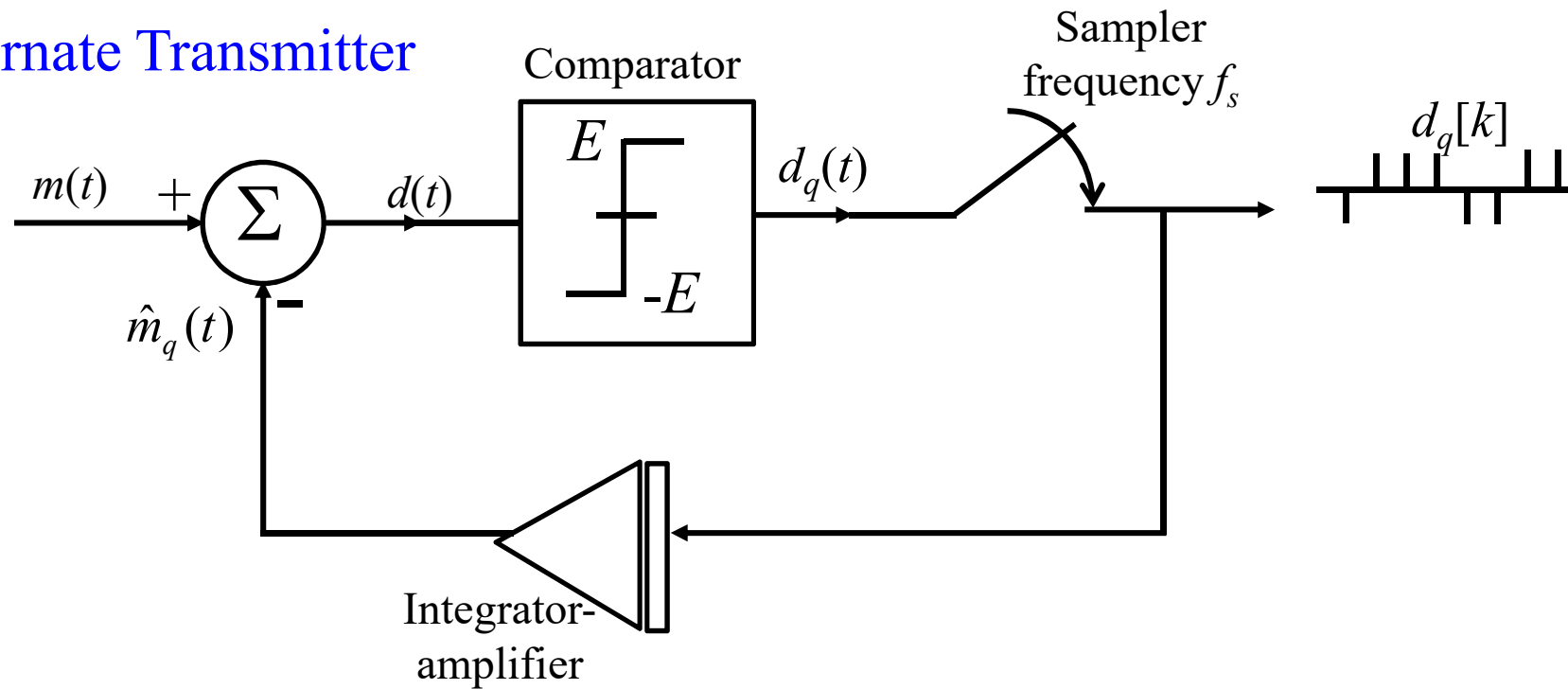


Receiver

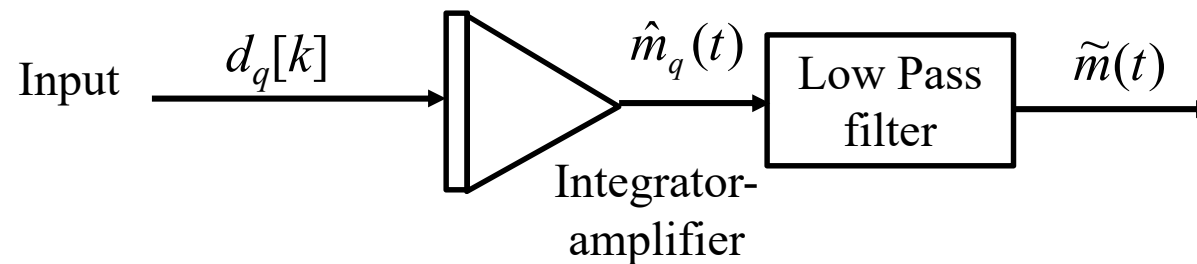


Delta Modulation

Alternate Transmitter

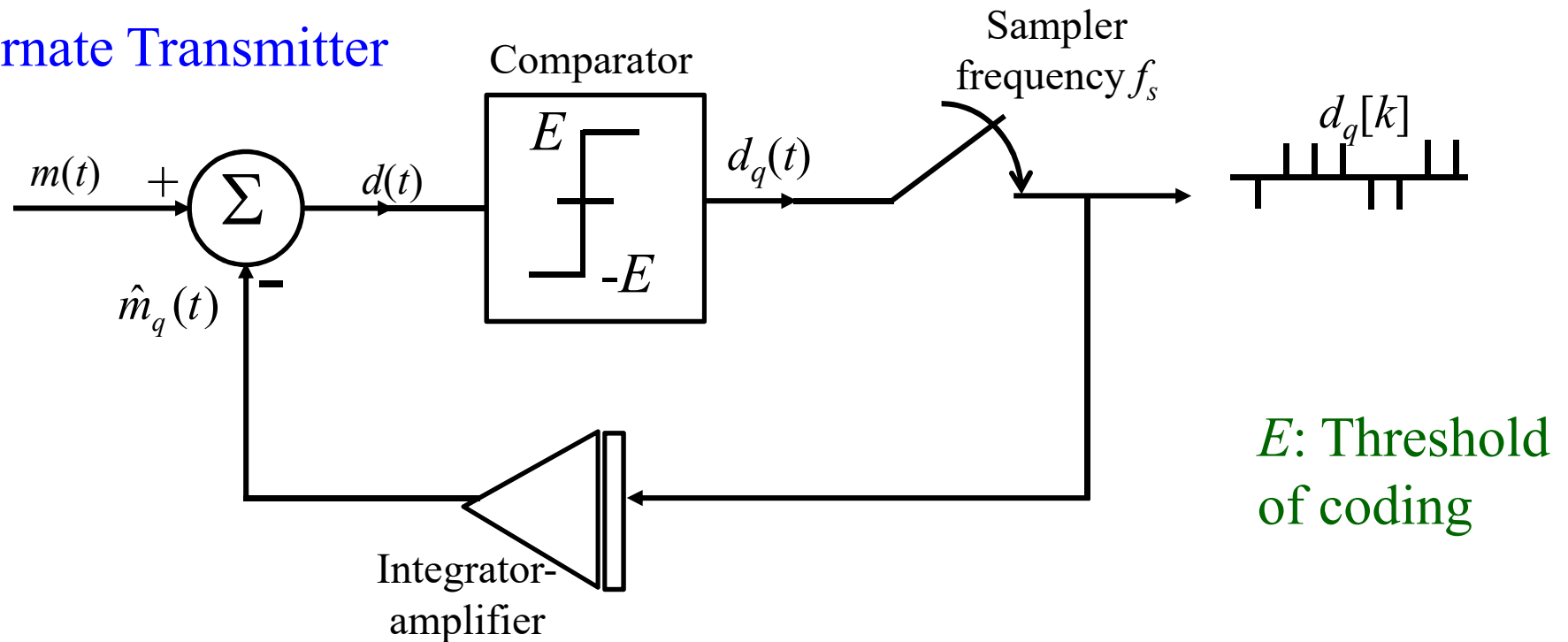


Alternate Receiver



Delta Modulation

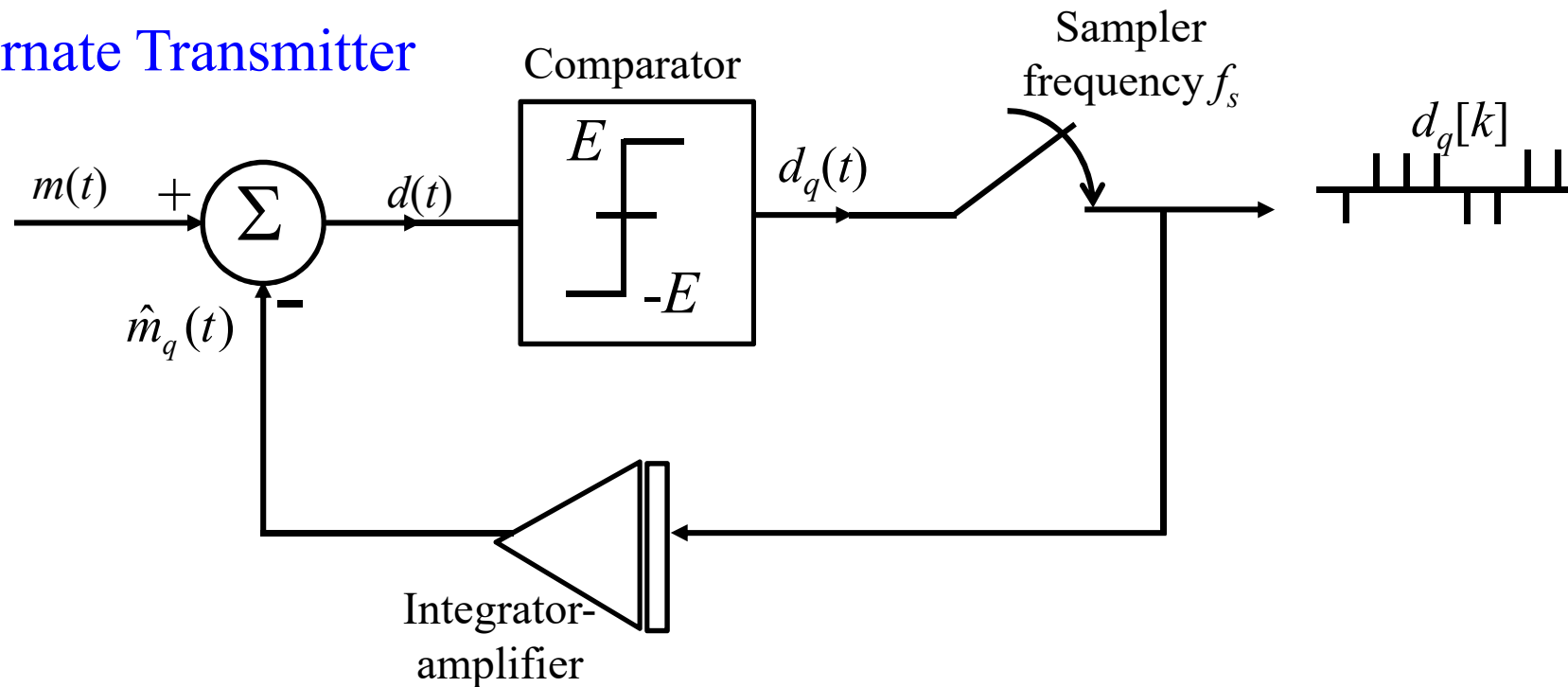
Alternate Transmitter



- Input $m(t)$ is analog
- $d(t)$ is compared in *comparator*: $d(t) > 0$? $d_q(t) = E : d_q(t) = -E$
- $d_q(t)$ is sampled @ f_s to generate $d_q[k]$
- $d_q[k]$'s are integrated and amplified to get $\hat{m}_q(t)$

Delta Modulation

Alternate Transmitter



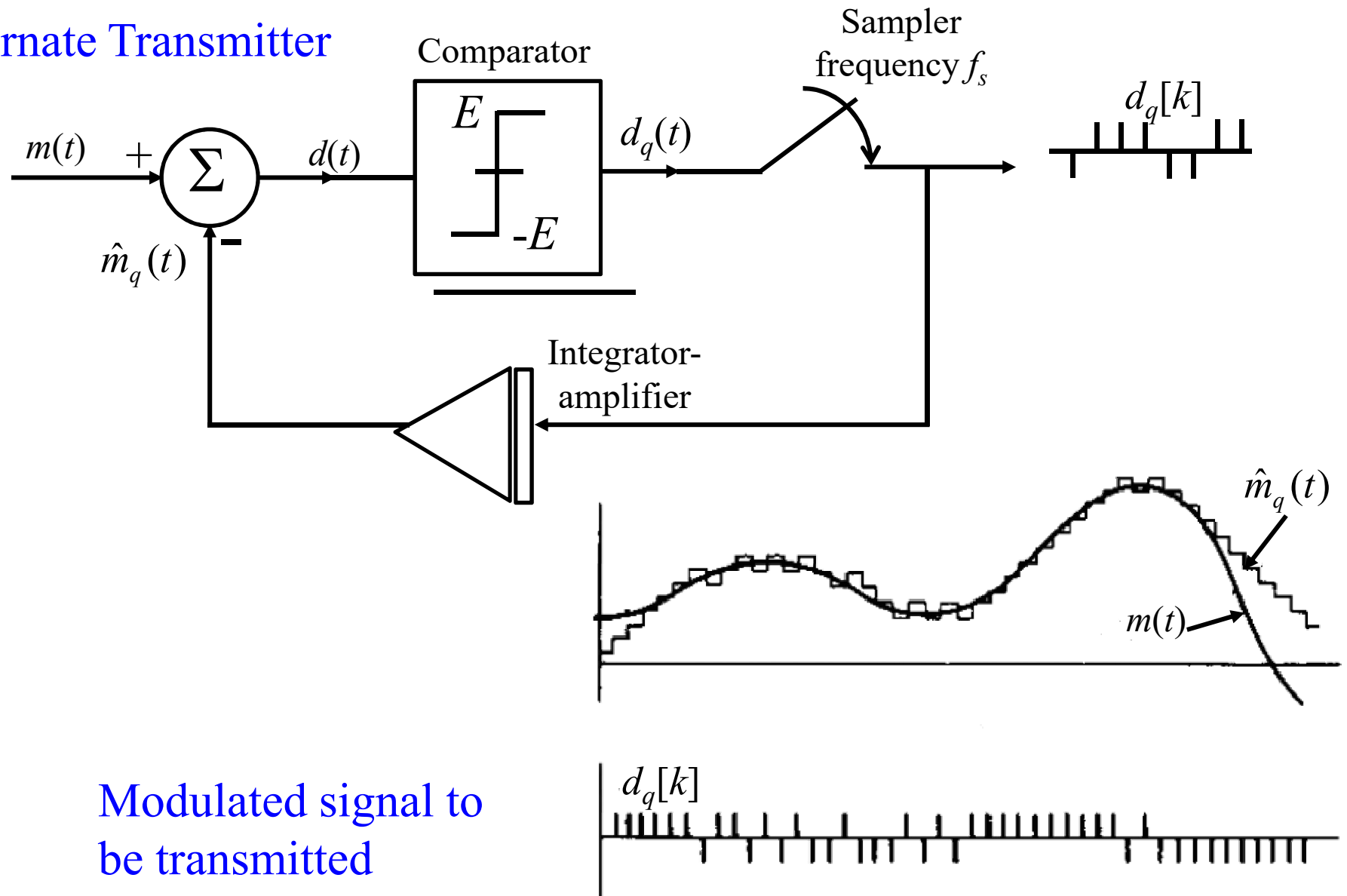
- The sampler generates pulse train $d_q[k]$ as follows

positive pulse if $m(t) > \hat{m}_q(t)$

negative pulse if $m(t) < \hat{m}_q(t)$

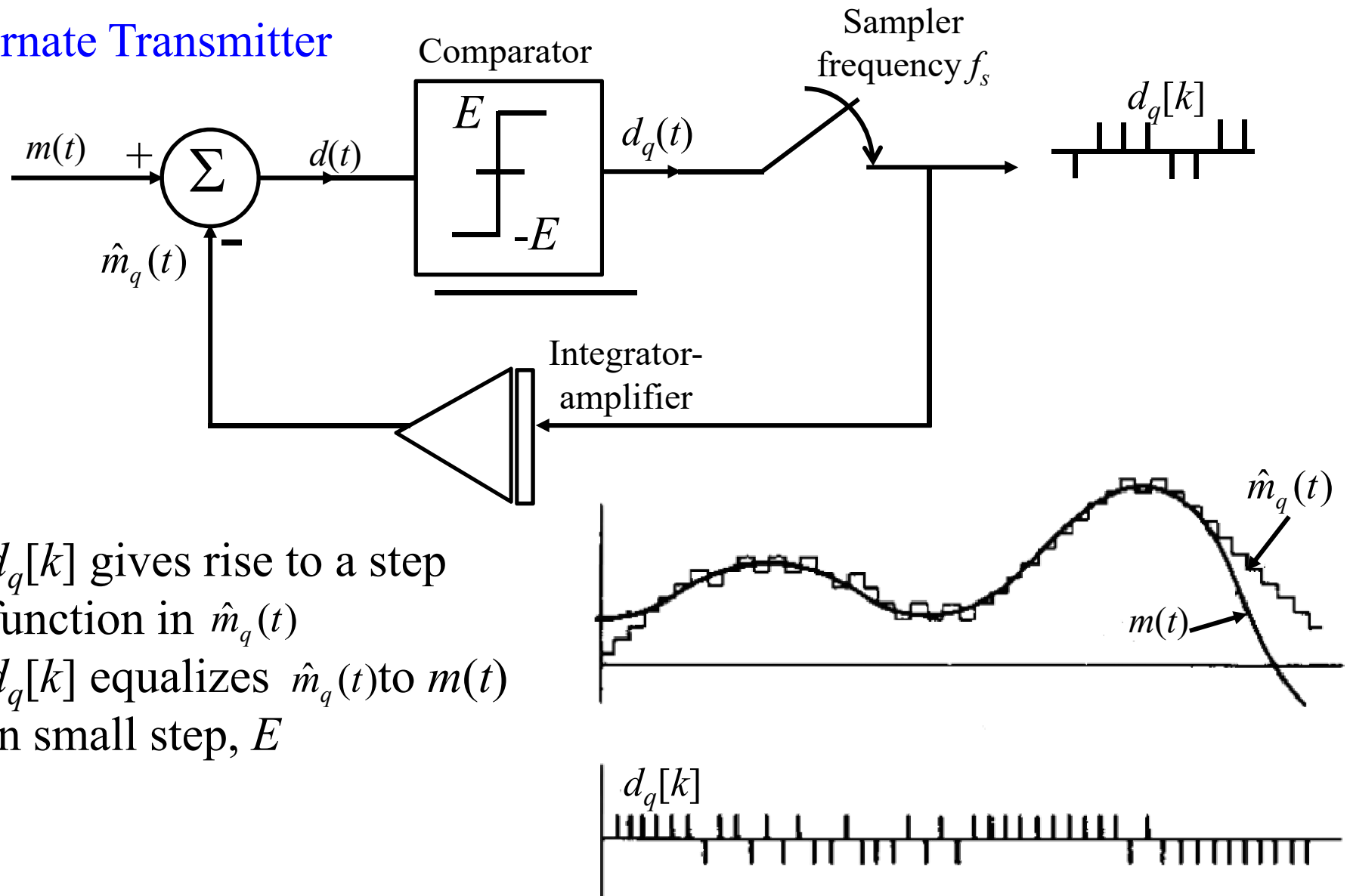
Delta Modulation

Alternate Transmitter



Delta Modulation

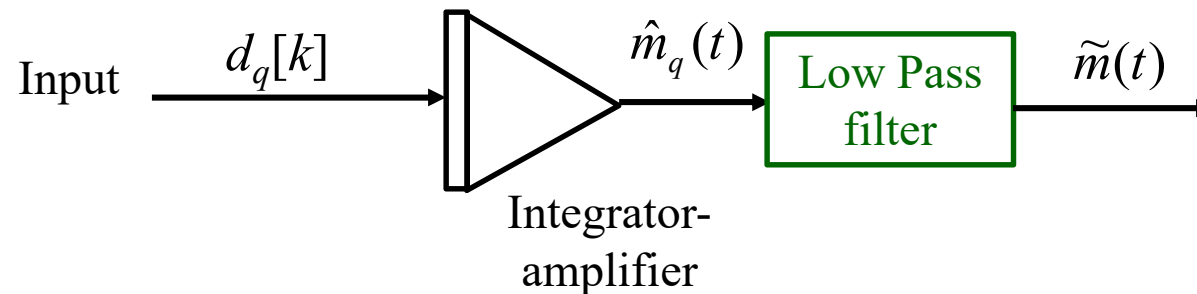
Alternate Transmitter



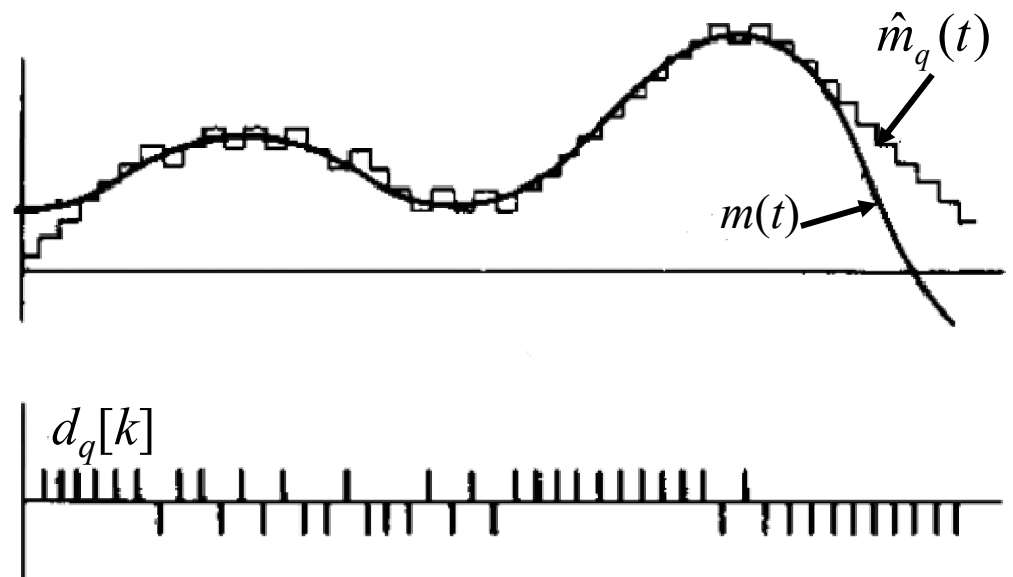
- $d_q[k]$ gives rise to a step function in $\hat{m}_q(t)$
- $d_q[k]$ equalizes $\hat{m}_q(t)$ to $m(t)$ in small step, E

Delta Modulation

Alternate Receiver

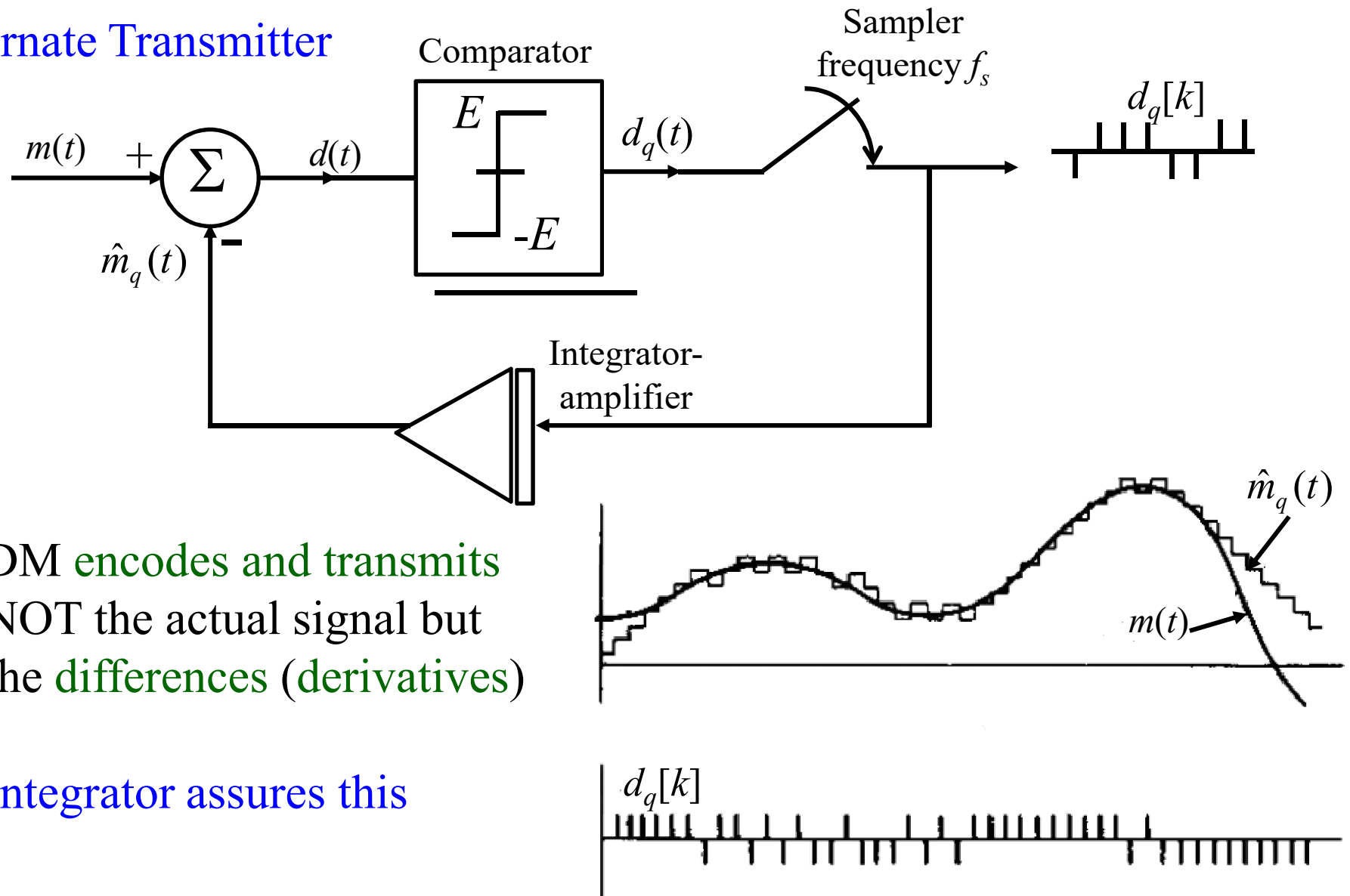


- Coarseness of $\hat{m}_q(t)$ is removed when passed through low pass filter at receiver



Delta Modulation

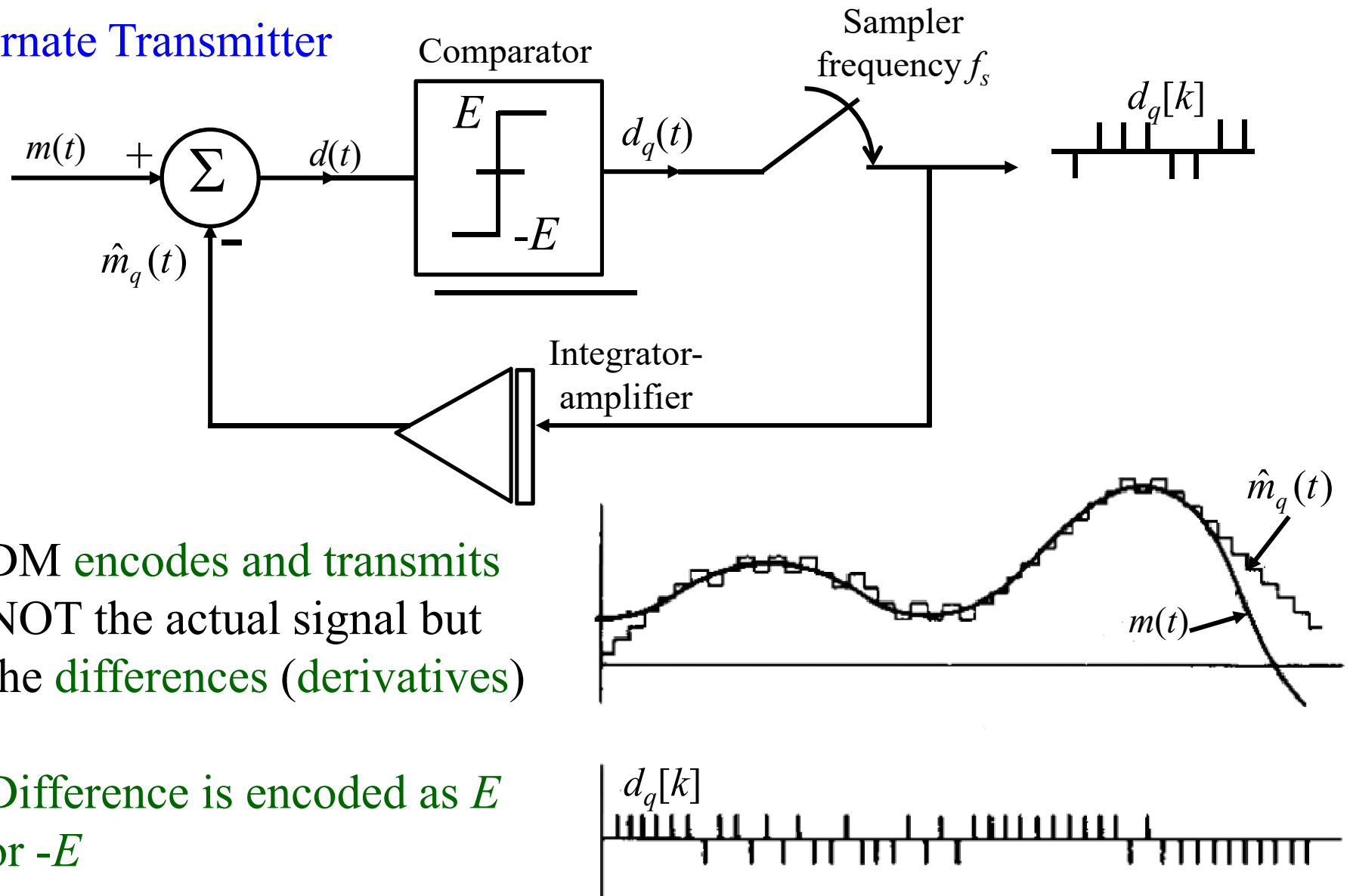
Alternate Transmitter



- DM **encodes and transmits** NOT the actual signal but the **differences (derivatives)**
- Integrator assures this

Delta Modulation

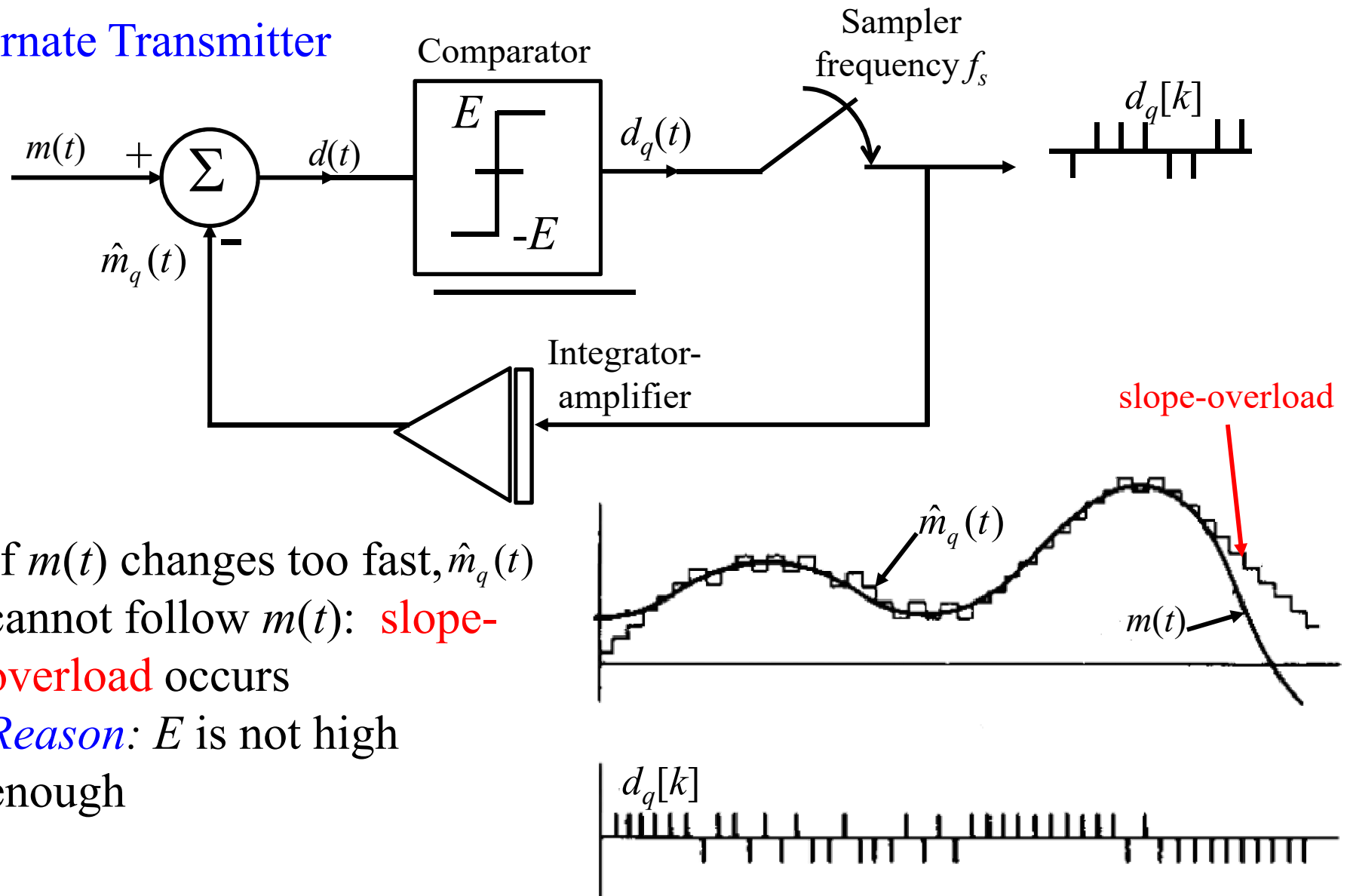
Alternate Transmitter



- DM encodes and transmits NOT the actual signal but the differences (derivatives)
- Difference is encoded as E or $-E$

Delta Modulation

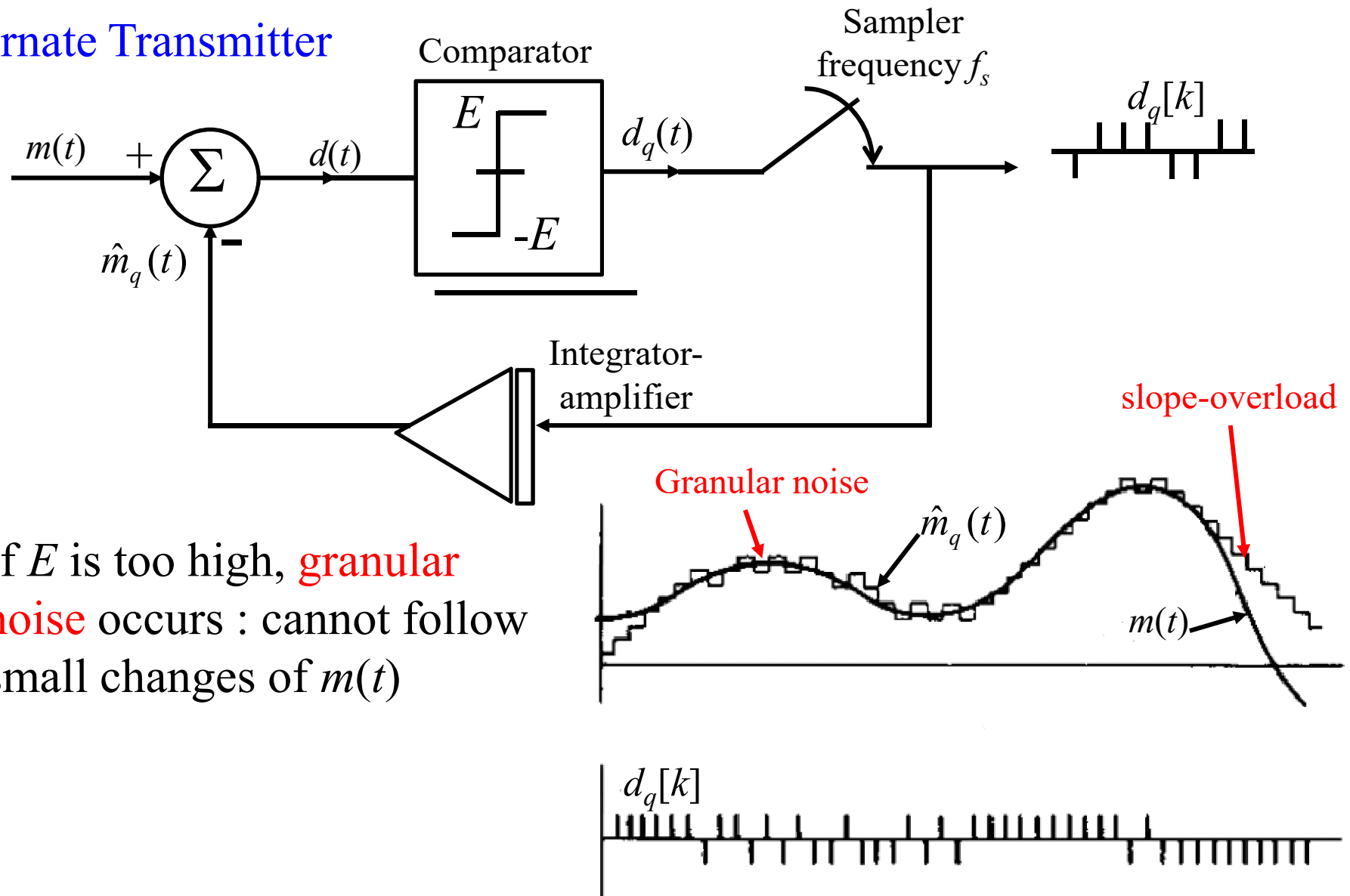
Alternate Transmitter



- If $m(t)$ changes too fast, $\hat{m}_q(t)$ cannot follow $m(t)$: **slope-overload** occurs
- *Reason*: E is not high enough

Delta Modulation

Alternate Transmitter

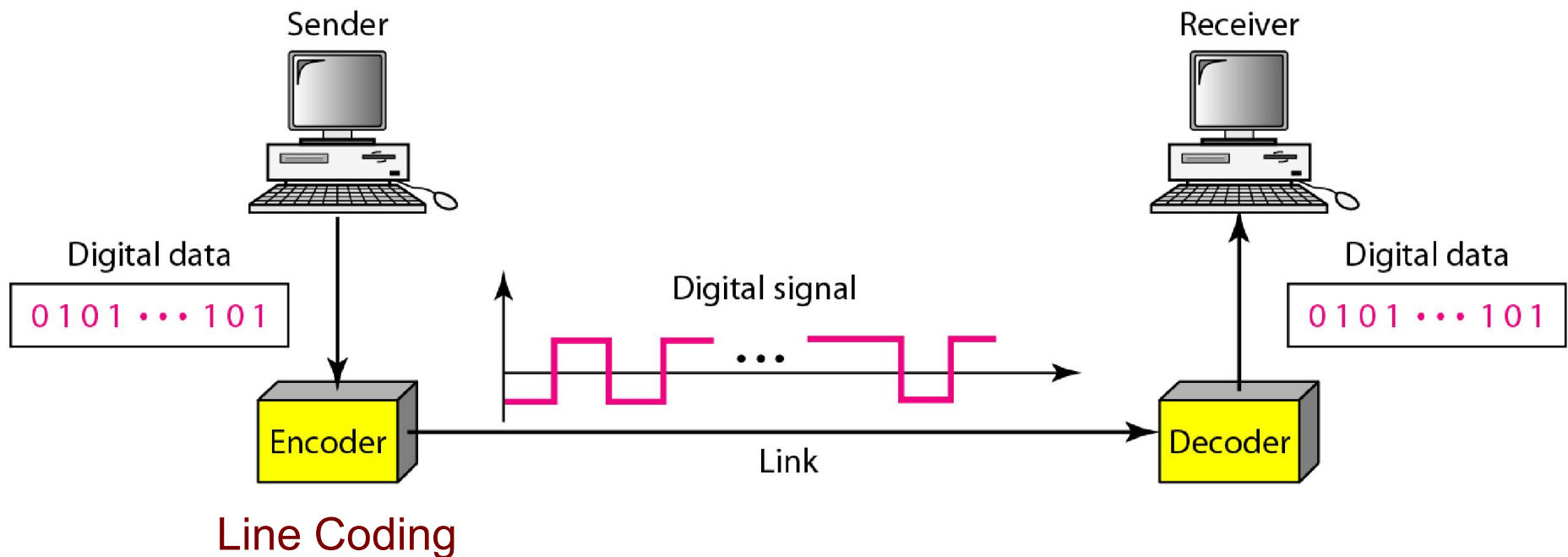


- If E is too high, **granular noise** occurs : cannot follow small changes of $m(t)$

Digital-to-Digital Conversion: **Line Coding, Block Coding and** **Scrambling**

From
Data Communications and Networking, 5th Edition
By Behrouz A. Forouzan

Line Coding and Decoding



- converts *digital data* elements to *digital signal* elements
- Example:
 $1 \rightarrow +V$
 $0 \rightarrow -V$

Data Element and Signal Element

- data element
 - smallest entity to represent a piece of information
 - 0, 1, etc,
- signal element
 - shortest unit of a digital signal
 - Each represents one or more data element

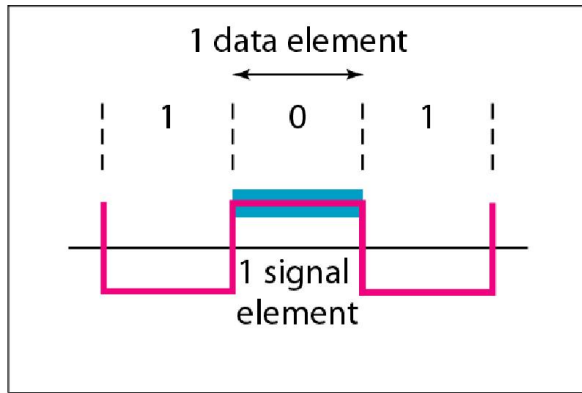
Data Element and Signal Element

- data element
 - smallest entity to represent a piece of information
 - 0, 1, etc,
- signal element
 - shortest unit of a digital signal
 - Each represents one or more data element
- signal element is carrier, data element is carried

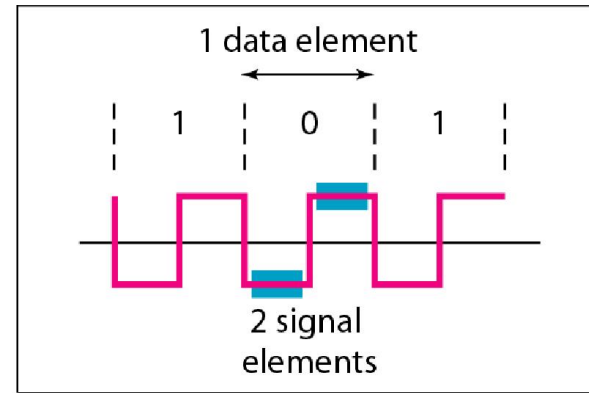
Data Element and Signal Element

- data element
 - smallest entity to represent a piece of information
 - 0, 1, etc,
- signal element
 - shortest unit of a digital signal
 - Each represents one or more data element
- signal element is carrier, data element is carried
- r = No. of data element is carried by each signal element

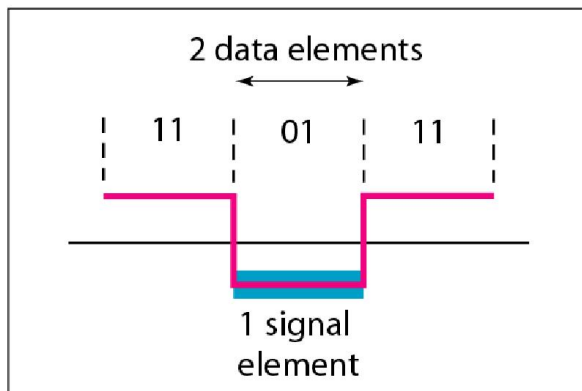
Data Element and Signal Element



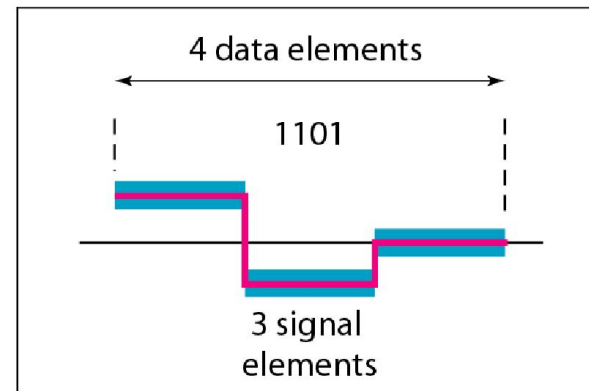
a. One data element per one signal element ($r = 1$)



b. One data element per two signal elements ($r = \frac{1}{2}$)



c. Two data elements per one signal element ($r = 2$)



d. Four data elements per three signal elements ($r = \frac{4}{3}$)

Data Rate and Signal Rate

- data rate, N
 - No. of data elements sent in 1 second
 - Other names: bit rate
- signal rate, S
 - No. of signal elements sent in 1 second
 - Other names: baud rate, pulse rate, modulation rate
- $r = N/S$ alternately, $S = N/r$

Data Rate and Signal Rate

- $r = N/S$ alternately, $S = N/r$
- Average Signal rate

$$S_{\text{ave}} = c \times N \times (1/r) \text{ baud}$$

Line Coding Guidelines

- r should be higher, $S = N/r$ should be lower
- Minimize *Baseline wandering*:
 - Running average of incoming signal power is *baseline*
 - Baseline is compared with incoming signal for decoding
 - Long runs of 0 or 1 drifts the baseline

Line Coding Guidelines

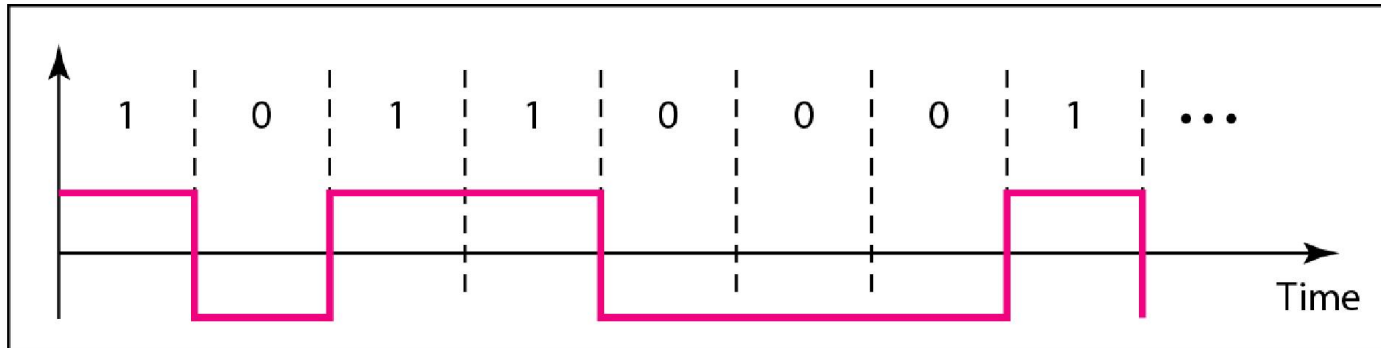
- Minimize *DC components*:
 - Constant voltage for long period
 - increaes low frequency components
 - bandpass channels **do NOT** support low frequencies

Line Coding Guidelines

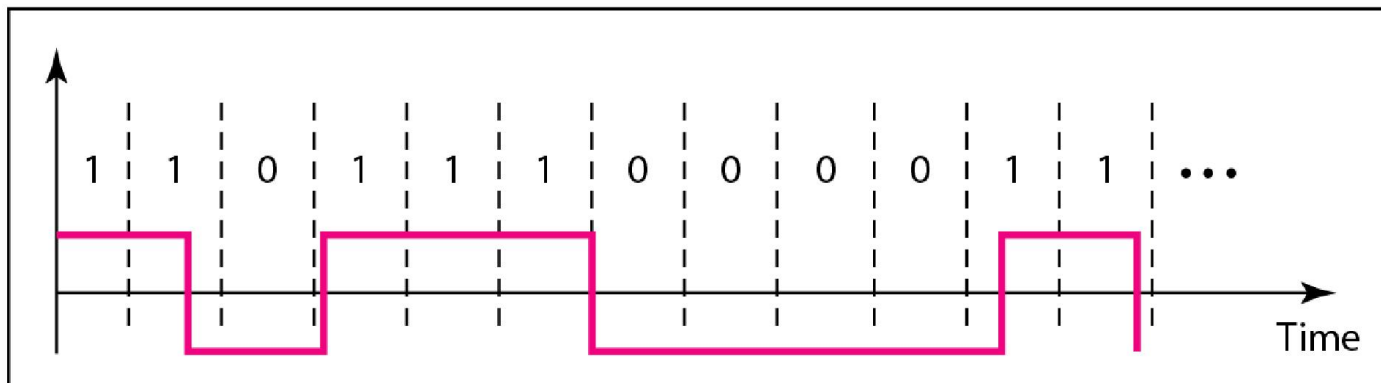
- Self synchronization
 - Sender's and receiver's clock must have same bit interval
 - Different intervals may misinterpret incoming signal

Line Coding Guidelines

Illustration of lack of synchronization



a. Sent



b. Received

Line Coding Guidelines

- Built-in Error Detection
 - Error occurs during transmission
 - Must be able to detect and/or correct them
 - Example: certain type of signal transition (change) is NOT part of the coding

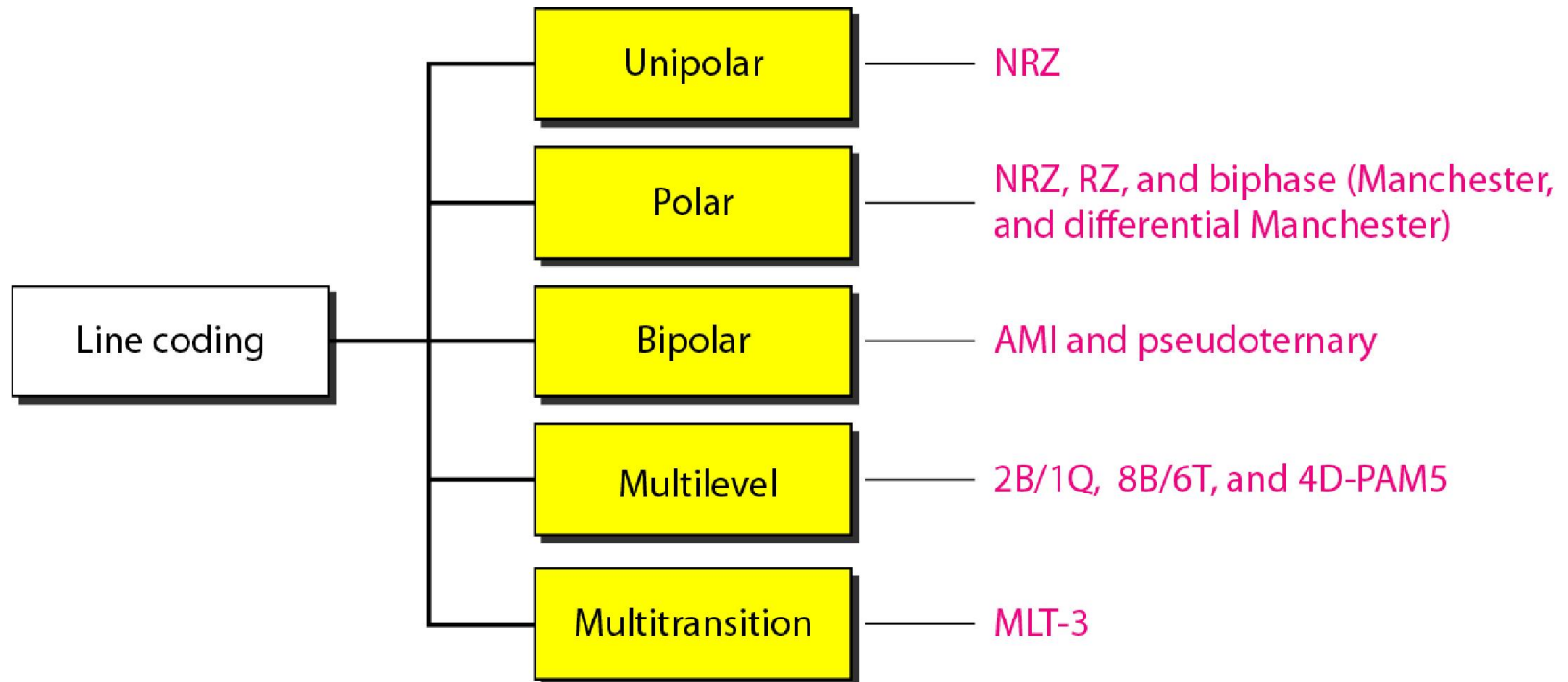
Line Coding Guidelines

- Higher Noise immunity
 - certain line coding prevents noises to be added to the signal
 - It is better than error detection

Line Coding Guidelines

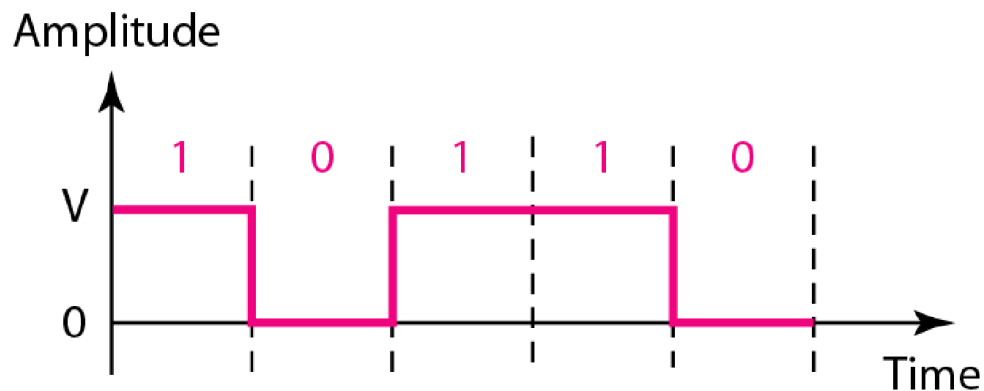
- Complexity
 - Simpler is better
 - A coding with FOUR signal levels is more difficult to interpret than one that uses TWO signal levels

Line Coding Schemes



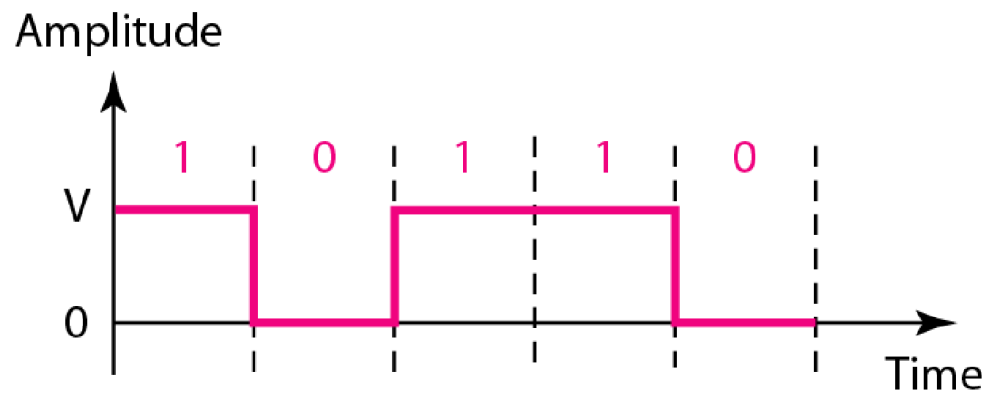
Unipolar

- All signal levels are on one side of the time axis - either above or below
- Example: Non Return to Zero (NRZ)
 - The signal level does not return to zero at middle of the bit



Unipolar: NRZ

- Pros
 - Simple
- Cons
 - Baseline wandering, DC components
 - No synchronization or error detection



$$r = 1$$

Polar

- signal levels are on BOTH sides of the time axis
- Polar NRZ
 - Uses two different voltage: $+V$ and $-V$
 - Two versions: NRZ-Level (NRZ-L) and NRZ-Inversion (NRZ-I)

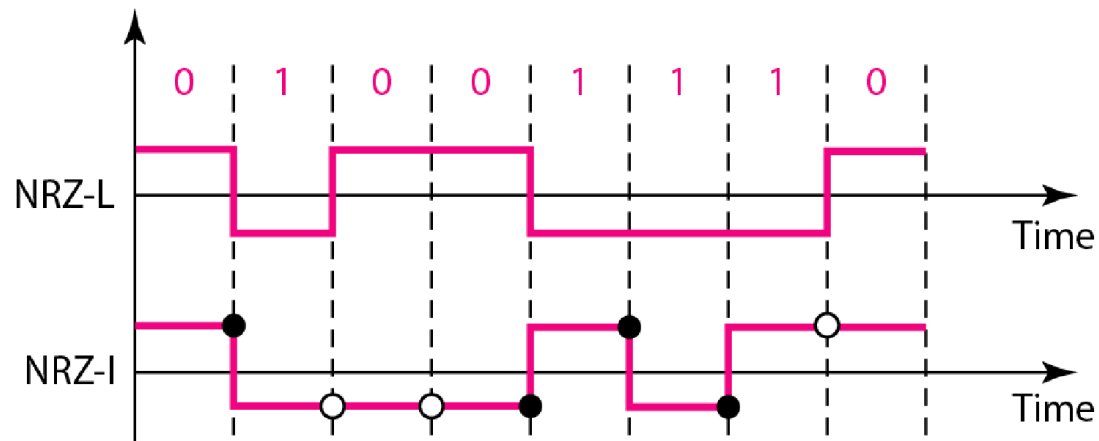
Polar: NRZ-L and NRZ-I

- Polar NRZ-L

$+V$: 0 and $-V$: 1

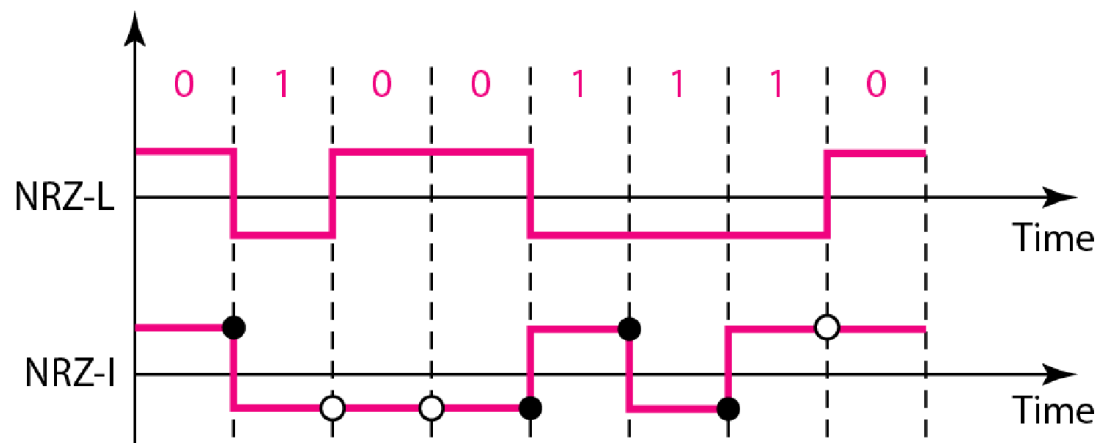
- Polar NRZ-I

Inversion : next bit is 1, No Inversion : next bit 0



Polar: NRZ-L and NRZ-I

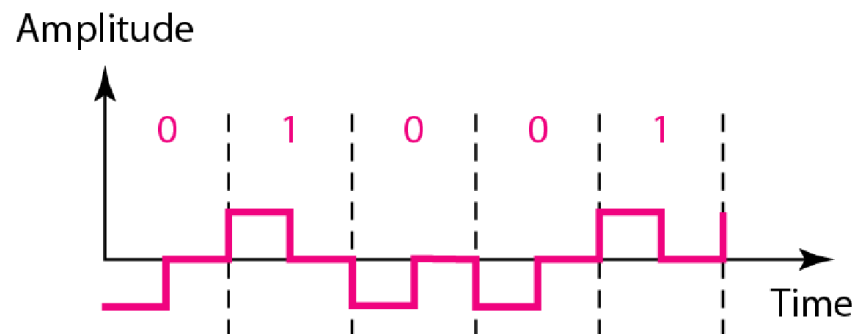
- Pros
 - Simple
- Cons
 - Baseline wandering, DC components, worse in NRZ-L
 - No synchronization or error detection



$$r = 1$$

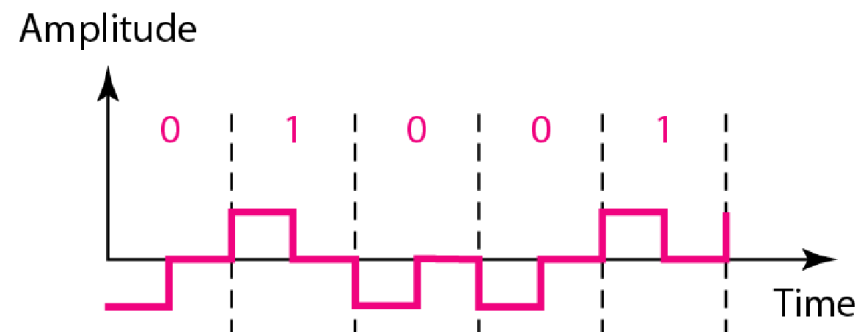
Polar: Return to Zero (RZ)

- 3 signal levels: $+V$, $-V$, 0
- Signal transition in the middle of the bit
 - bit 0: $-V$ to 0
 - bit 1: $+V$ to 0



Polar: Return to Zero (RZ)

- Pros
 - NO Baseline wandering, NO DC components
 - Self synchronization: transition synchronizes both sender and receiver
- Cons
 - More complex: 3 voltage levels to identify
 - No error detection



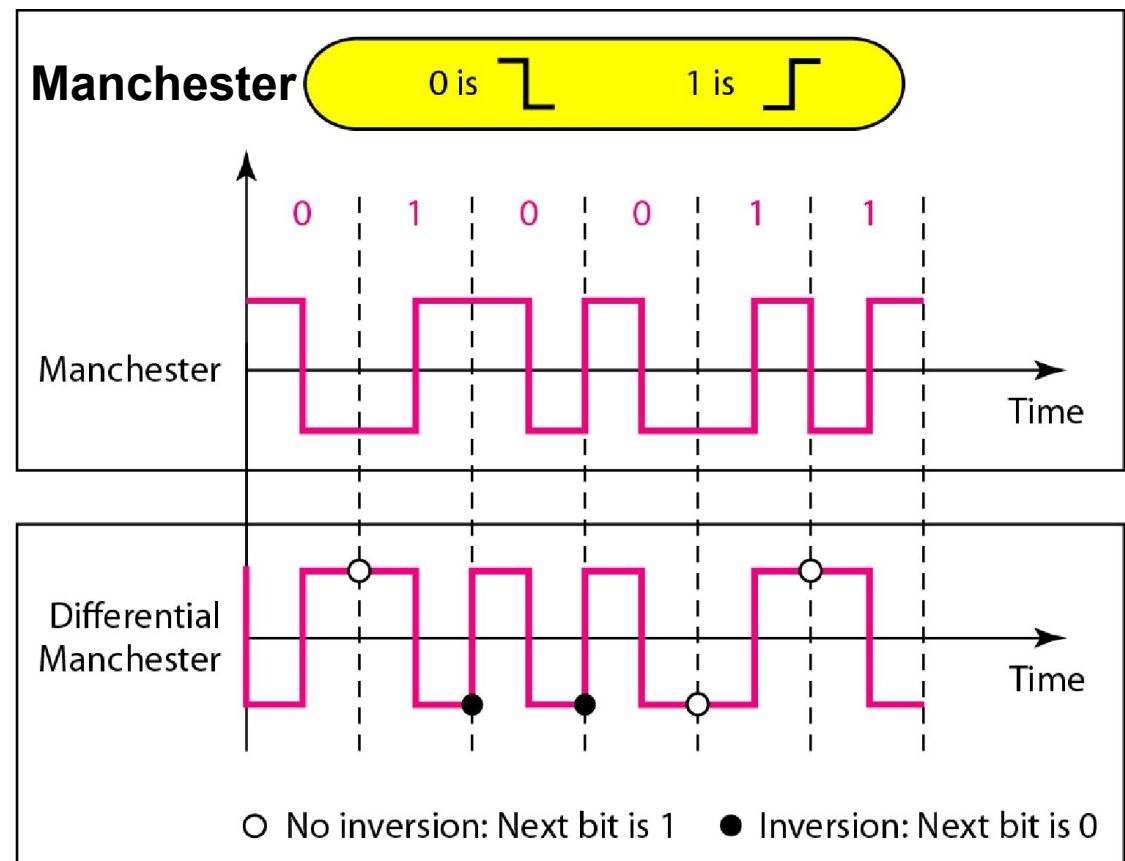
$$r = 1/2$$

Polar: Biphase-Manchester and Differential Manchester

- 2 signal levels: $+V$, $-V$
- Signal transition *always* in the middle of the bit, *similar to RZ*
- Manchester
 - Combines NRZ-L and RZ
- Differential Manchester
 - Combines NRZ-I and RZ

Polar: Biphase-Manchester and Differential Manchester

- Manchester
 - Level determines bits
- Differential Manchester
 - Inversion/no inversion determines bits



Polar: Biphase-Manchester and Differential Manchester

- Pros
 - 2 signal elements
 - NO Baseline wandering, NO DC components
 - Self synchronization
- Cons
 - $r = 1/2$
 - No error detection

